



香港科技大學  
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

# **ELEC 3310**

# **Digital Fundamentals and System Design**

**Wenchao Miao**

# Outline Lecture 11 Finite State Machines Latches; Flip-flops; & FPGA



THE HONG KONG  
UNIVERSITY OF SCIENCE  
AND TECHNOLOGY

**1 Basic Concepts of Latches**

**2 SR Latch**

**3 Application: Contact-Bounce Eliminator**

**4 Flip-flop**

**5 Programmable Logic Devices**

**6 Field-Programmable Gate Arrays**

## Main Content

1. Learning Outcomes
2. Basic Concepts of Latches

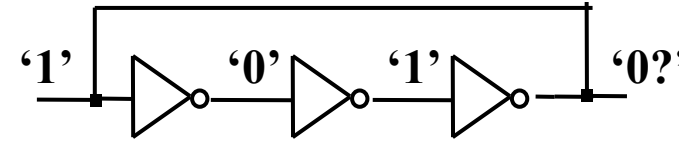
- 1. Use logic gates to construct basic latches**
- 2. Explain the operation of a basic S-R latch**
- 3. Explain the operation of a gated S-R latch**
- 4. Explain the operation of a gated D latch**
- 5. Explain the difference between an S-R latch and a D latch**
- 6. Recognize the difference between a latch and a flip-flop**
- 7. Construct D and JK flip-flops based on latches**
- 8. Describe the basic structure of programmable logic devices (PLDs)**



- **Sequential Logic** is the **combinational Logic** with a **FEEDBACK** path
- Feedback path connect output back to input
- Output in **combinational logic** depends **ONLY** on the combination of **CURRENT** input bits
- Output in **sequential logic** depends on the combination of **CURRENT** input bits **AND** the sequence of **PAST** input bits (history!)
- So **sequential logic** **REMEMBERS** sequence of past input bits and hence has a **MEMORY**!

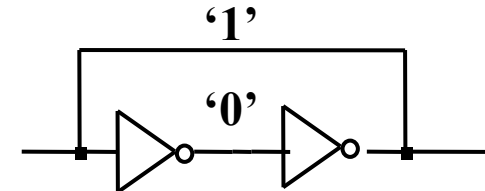
- Odd number of inverters with feedback

**Unstable: keep oscillating  
between 0 and 1**



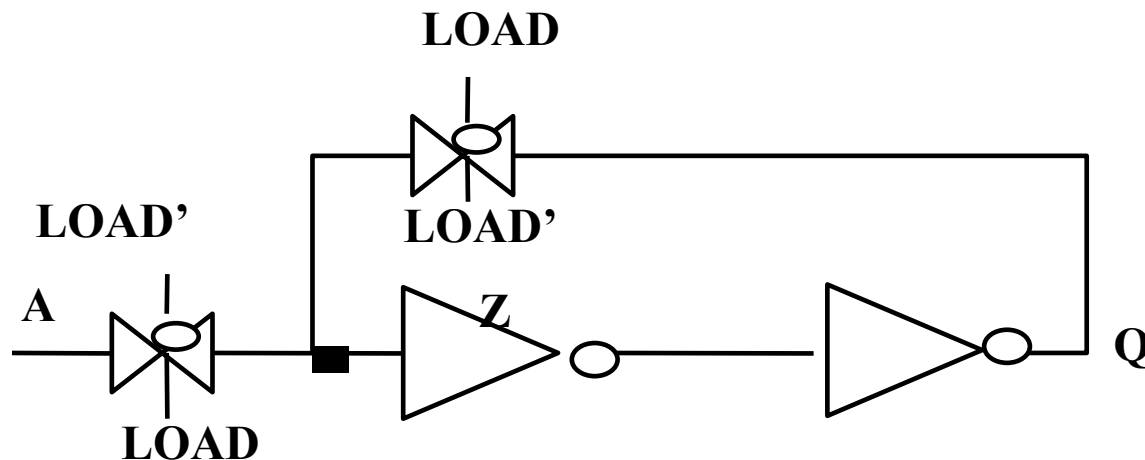
- Two inverters with feedback
  - Memory element (two stable states: 0 and 1)
  - Read-only, but has no write function

**Memory: The bit “0” is stored  
between the two inverters**



# Inverters with Feedback: Memory!

- Memory with READ/WRITE capability
  - **Selectively break the feedback path** by transmission gates to load new value into the cell
  - **WRITE** stage: A can be written to Z when **LOAD = 1**
    - **Feedback is broken**
  - **READ** stage: Z holds the value when **LOAD = 0**
    - **Feedback is ON** (Input access is broken!)

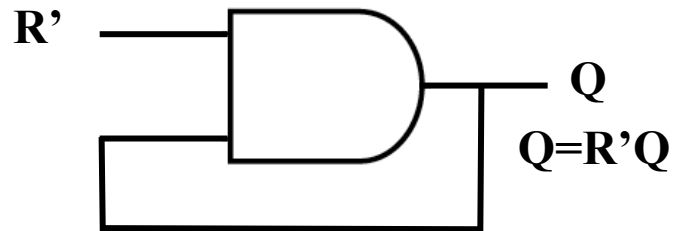


This circuit is called “D-type latch”  
“transparent D-latch” or just “D-latch”

# AND/OR Gates with a feedback

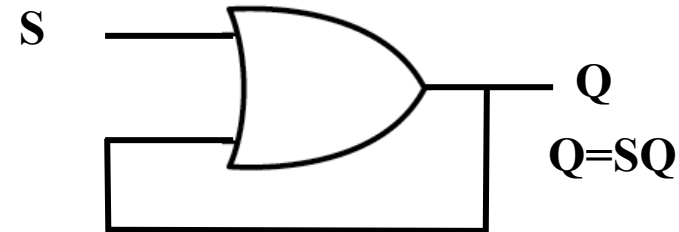


THE HONG KONG  
UNIVERSITY OF SCIENCE  
AND TECHNOLOGY



| R' | Q | State  |
|----|---|--------|
| 0  | 0 | RESET  |
| 1  | Q | Memory |

- Once RESET, it cannot be SET again

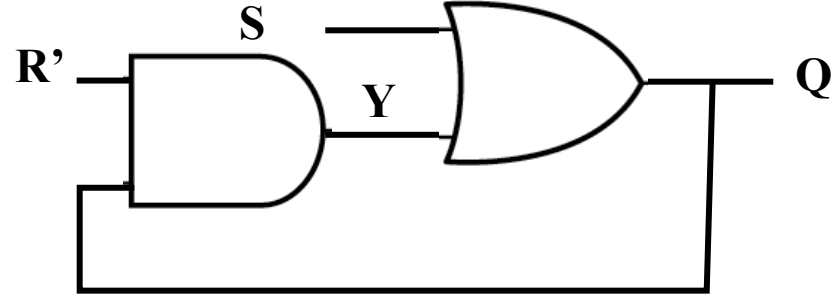
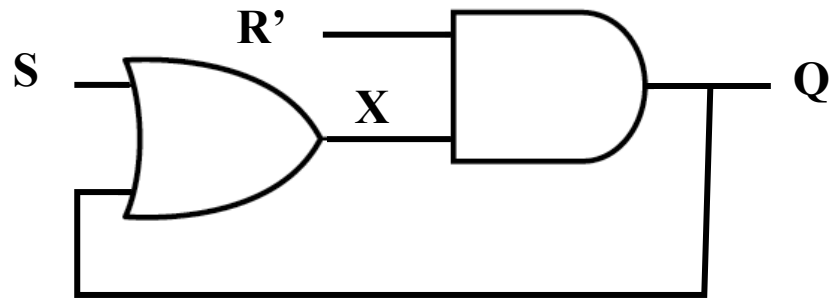
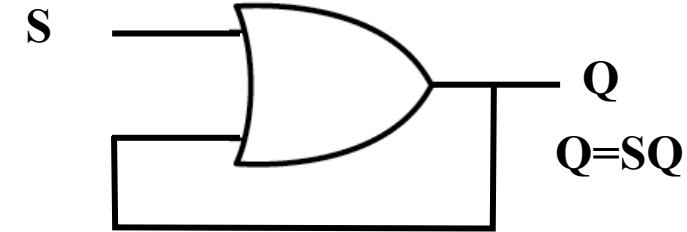
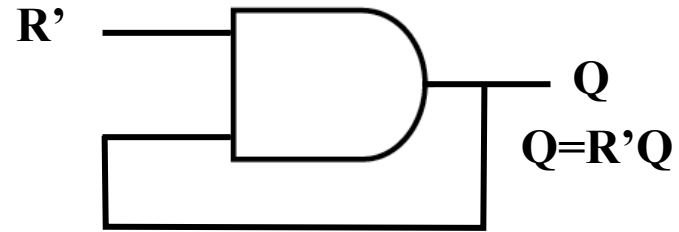


| S | Q | State  |
|---|---|--------|
| 0 | Q | Memory |
| 1 | 1 | SET    |

- ▶ Once SET, it cannot be RESET again

State: state of output  
SET means  $Q = 1$   
RESET means  $Q = 0$

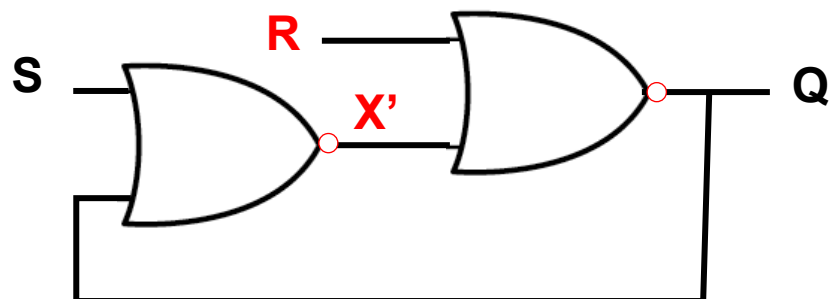
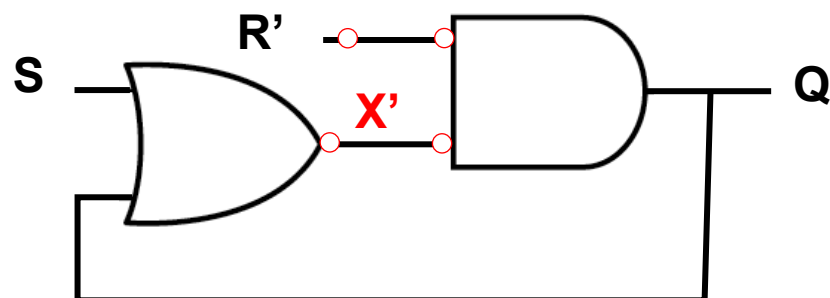
# Combine AND/OR Gates with a feedback



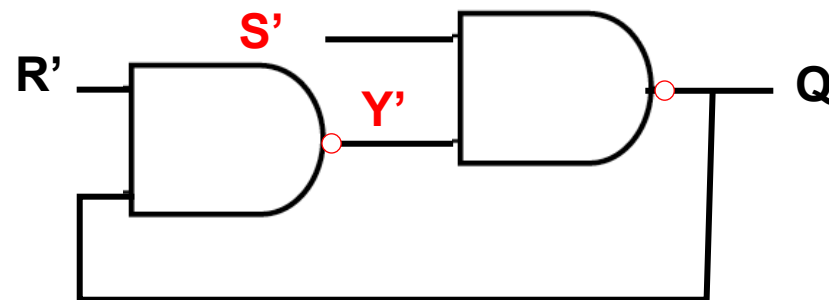
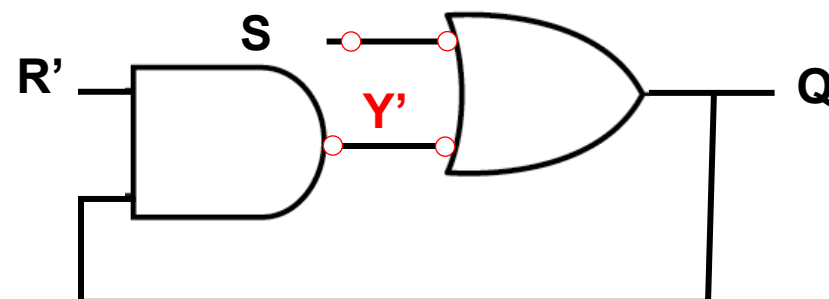
| R' | S | X | Q | State  |
|----|---|---|---|--------|
| 0  | 0 | 0 | 0 | Reset  |
| 0  | 1 | 1 | 0 | Reset  |
| 1  | 0 | Q | Q | Memory |
| 1  | 1 | 1 | 1 | Set    |

| S | R' | Y | Q | State  |
|---|----|---|---|--------|
| 0 | 0  | 0 | 0 | Reset  |
| 0 | 1  | Q | Q | Memory |
| 1 | 0  | 0 | 1 | Set    |
| 1 | 1  | 1 | 1 | Set    |

# NAND/NOR Conversion

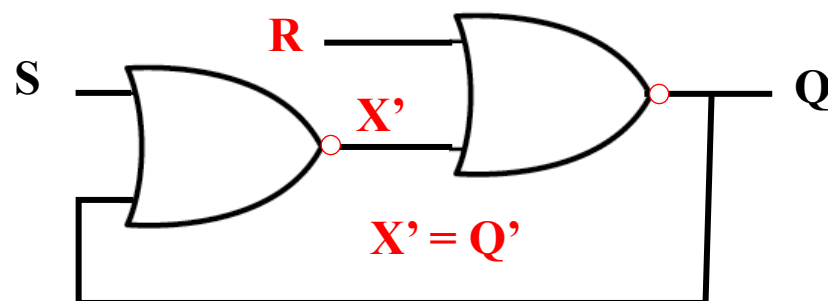


| R' | S | X | Q | State  |
|----|---|---|---|--------|
| 0  | 0 | 0 | 0 | Reset  |
| 0  | 1 | 1 | 0 | Reset  |
| 1  | 0 | Q | Q | Memory |
| 1  | 1 | 1 | 1 | Set    |



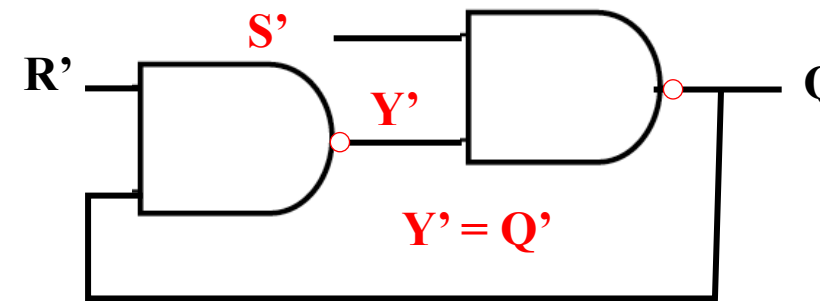
| S | R' | Y | Q | State  |
|---|----|---|---|--------|
| 0 | 0  | 0 | 0 | Reset  |
| 0 | 1  | Q | Q | Memory |
| 1 | 0  | 0 | 1 | Set    |
| 1 | 1  | 1 | 1 | Set    |

# NAND/NOR Conversion



| R' | S | X | Q | State  |
|----|---|---|---|--------|
| 0  | 0 | 0 | 0 | Reset  |
| 0  | 1 | 1 | 0 | Reset  |
| 1  | 0 | Q | Q | Memory |
| 1  | 1 | 1 | 1 | Set    |

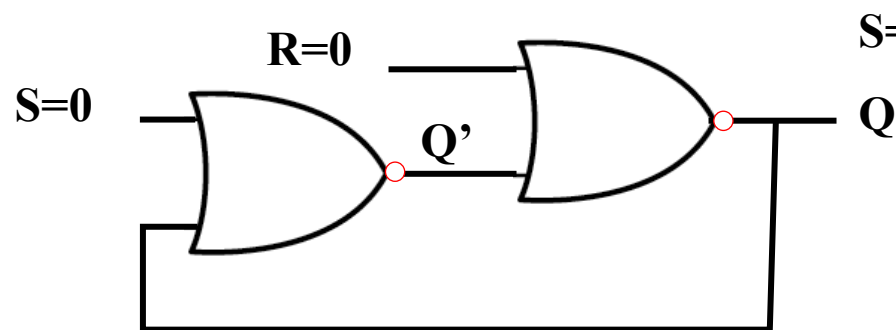
| R | S | X' | Q | State     |
|---|---|----|---|-----------|
| 0 | 0 | Q' | Q | Memory    |
| 0 | 1 | 0  | 1 | Set       |
| 1 | 0 | 1  | 0 | Reset     |
| 1 | 1 | 0  | 0 | Forbidden |



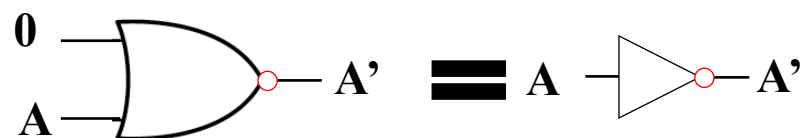
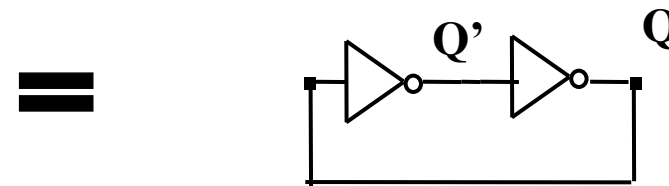
| S | R' | Y | Q | State  |
|---|----|---|---|--------|
| 0 | 0  | 0 | 0 | Reset  |
| 0 | 1  | Q | Q | Memory |
| 1 | 0  | 0 | 1 | Set    |
| 1 | 1  | 1 | 1 | Set    |

| S' | R' | Y' | Q | State     |
|----|----|----|---|-----------|
| 0  | 0  | 1  | 1 | Forbidden |
| 0  | 1  | 0  | 1 | Set       |
| 1  | 0  | 1  | 0 | Reset     |
| 1  | 1  | Q' | Q | Memory    |

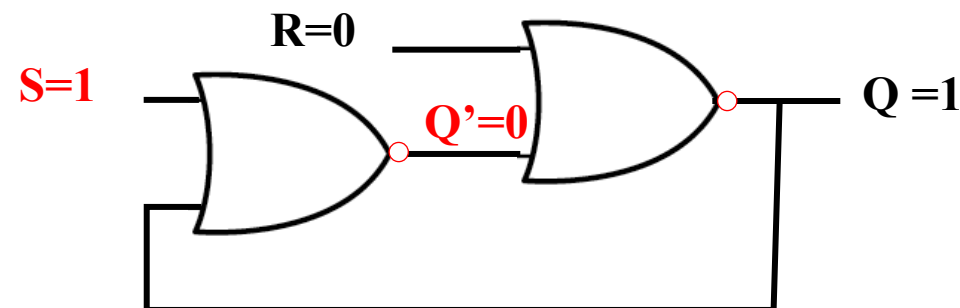
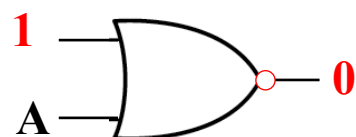
# NOR Conversion – SR Latch



$S=0$ ;  $R=0$ ; Memory  $\rightarrow$  Keeping the bit  $Q$

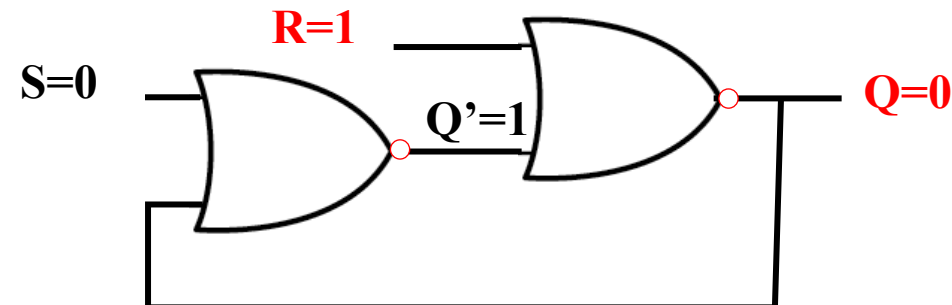


$S=1$ ;  $R=0$ ; SET  $\rightarrow$  LOAD '1' into MEMORY



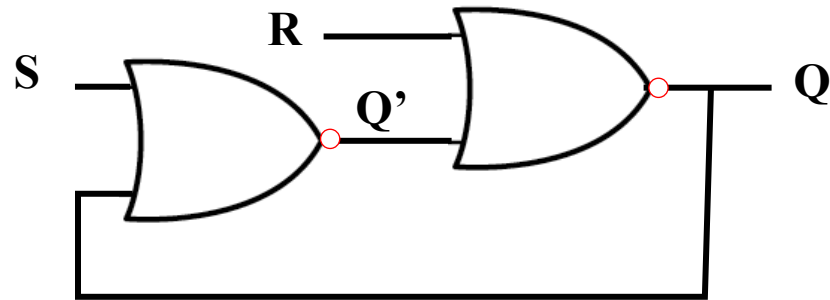
| R | S | $Q'$ | Q | State     |
|---|---|------|---|-----------|
| 0 | 0 | $Q'$ | Q | Memory    |
| 0 | 1 | 0    | 1 | Set       |
| 1 | 0 | 1    | 0 | Reset     |
| 1 | 1 | 0    | 0 | Forbidden |

$S=0$ ;  $R=1$ ; RESET  $\rightarrow$  LOAD '0' into MEMORY

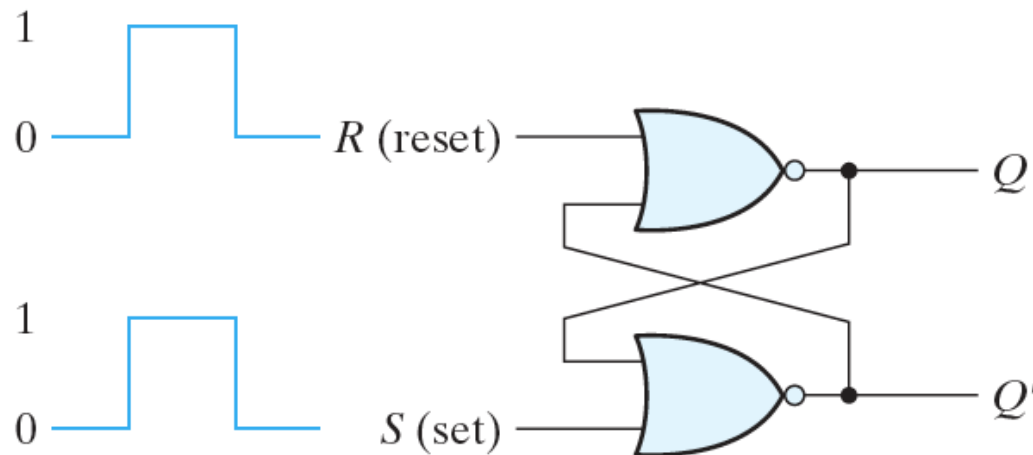




# SR Latch: Cross-coupled NOR gate



| R | S | Q' | Q | State     |
|---|---|----|---|-----------|
| 0 | 0 | Q' | Q | Memory    |
| 0 | 1 | 0  | 1 | Set       |
| 1 | 0 | 1  | 0 | Reset     |
| 1 | 1 | 0  | 0 | Forbidden |

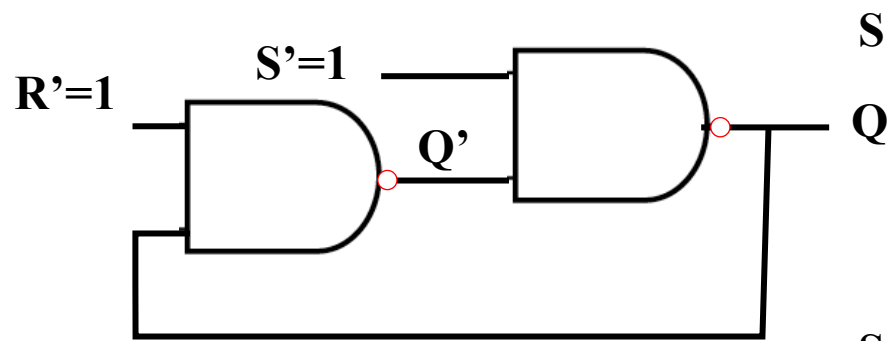


(a) Logic diagram

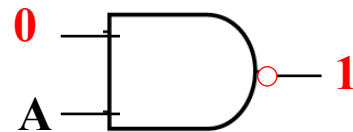
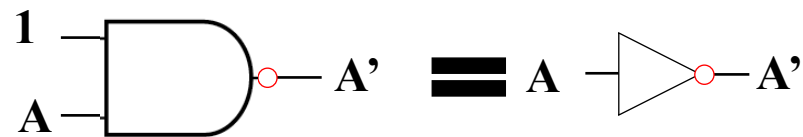
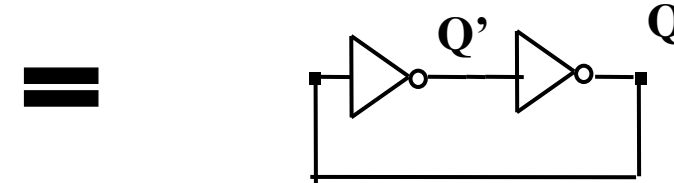
| $S$ | $R$ | $Q$ | $Q'$                      |
|-----|-----|-----|---------------------------|
| 1   | 0   | 1   | 0                         |
| 0   | 0   | 1   | 0 (after $S = 1, R = 0$ ) |
| 0   | 1   | 0   | 1                         |
| 0   | 0   | 0   | 1 (after $S = 0, R = 1$ ) |
| 1   | 1   | 0   | 0 (forbidden)             |

(b) Function table

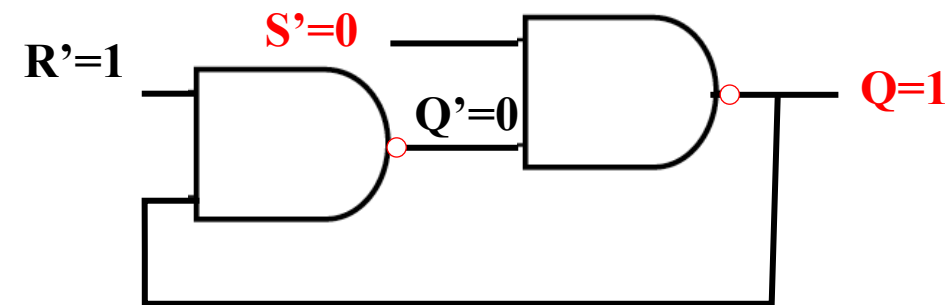
# NAND Conversion – S' R' Latch



$S'=1$ ;  $R'=1$ ; Memory  $\rightarrow$  Keeping the bit  $Q$

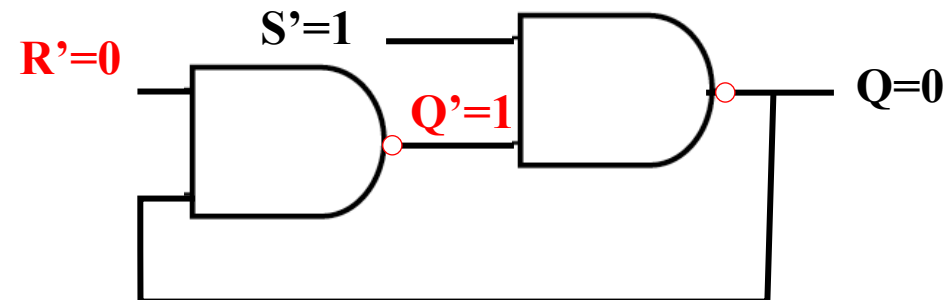


$S'=0$ ;  $R'=1$ ; SET  $\rightarrow$  LOAD '1' into MEMORY

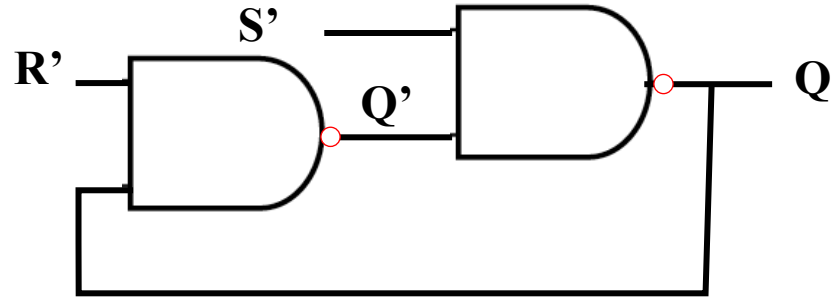


| $S'$ | $R'$ | $Q'$ | $Q$ | State     |
|------|------|------|-----|-----------|
| 0    | 0    | 1    | 1   | Forbidden |
| 0    | 1    | 0    | 1   | Set       |
| 1    | 0    | 1    | 0   | Reset     |
| 1    | 1    | $Q'$ | $Q$ | Memory    |

$S'=1$ ;  $R'=0$ ; RESET  $\rightarrow$  LOAD '0' into MEMORY

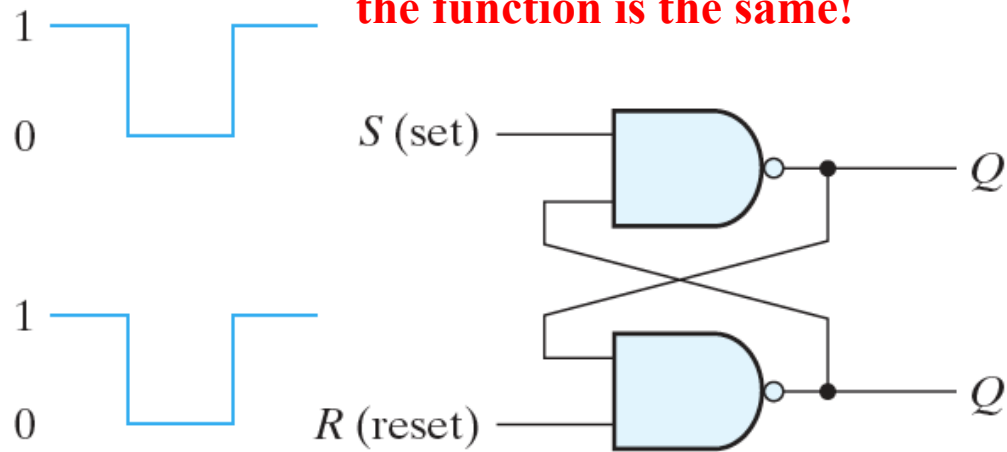


# $S'$ $R'$ Latch: Cross-coupled NAND gate



| $S'$ | $R'$ | $Q'$ | $Q$ | State            |
|------|------|------|-----|------------------|
| 0    | 0    | 1    | 1   | <b>Forbidden</b> |
| 0    | 1    | 0    | 1   | <b>Set</b>       |
| 1    | 0    | 1    | 0   | <b>Reset</b>     |
| 1    | 1    | $Q'$ | $Q$ | <b>Memory</b>    |

It doesn't matter what you call the inputs ( $S'$  vs  $S$  or  $R'$  vs  $R$ ), the function is the same!



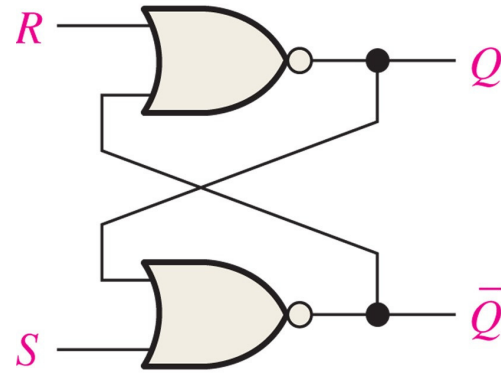
(a) Logic diagram

| $S$ | $R$ | $Q$ | $Q'$ |                         |
|-----|-----|-----|------|-------------------------|
| 1   | 0   | 0   | 1    |                         |
| 1   | 1   | 0   | 1    | (after $S = 1, R = 0$ ) |
| 0   | 1   | 1   | 0    |                         |
| 1   | 1   | 1   | 0    | (after $S = 0, R = 1$ ) |
| 0   | 0   | 1   | 1    | (forbidden)             |

(b) Function table

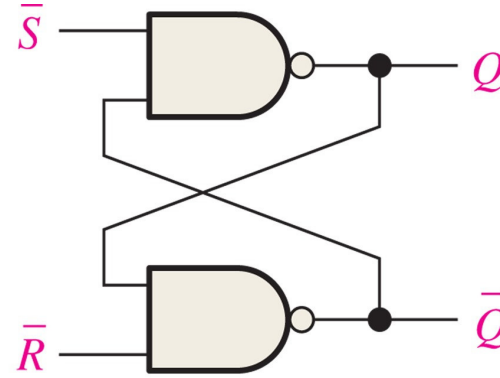
# S-R (SET-RESET) Latches (2 versions)

Cross-coupled NOR gates



(a) Active-HIGH input S-R latch

Cross-coupled NAND gates



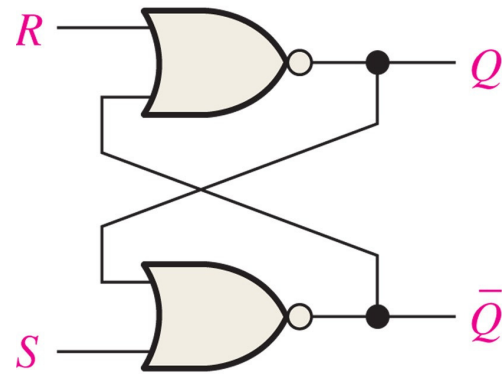
(b) Active-LOW input  $\bar{S}$ - $\bar{R}$  latch

- A latch is a type of bistable logic device or multivibrator
- Notice that the output of each gate is connected to an input of the opposite gate. This produces the regenerative feedback that is characteristic of all latches and flip-flop

# S-R (SET-RESET) Latches (2 versions)

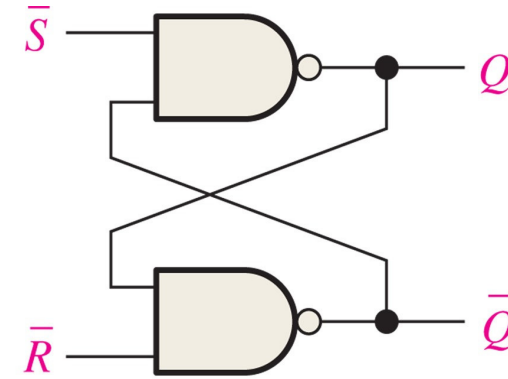


Cross-coupled NOR gates



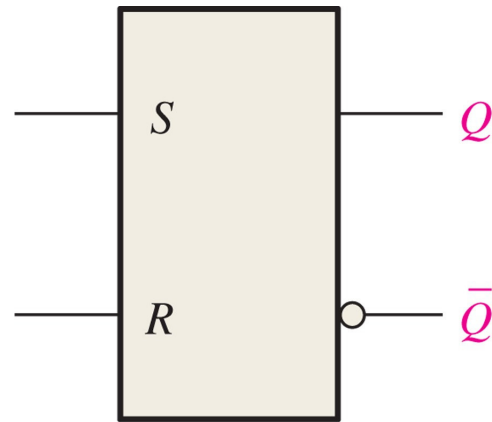
(a) Active-HIGH input S-R latch

Cross-coupled NAND gates

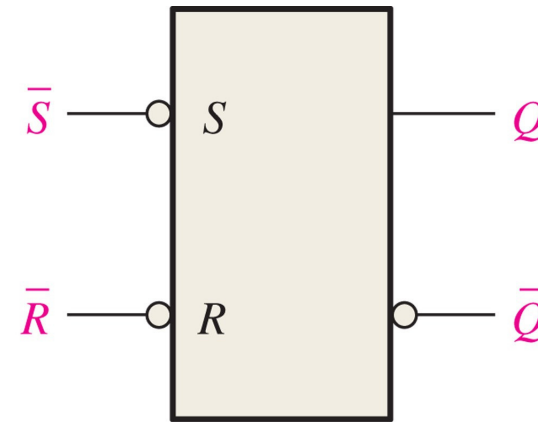


(b) Active-LOW input  $\bar{S}$ - $\bar{R}$  latch

Logic symbols:

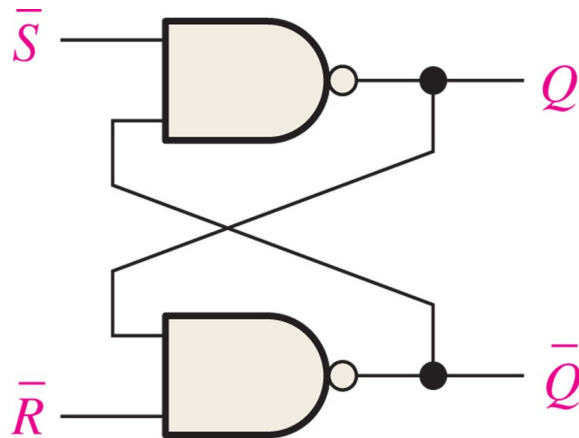


(a) Active-HIGH input  
S-R latch

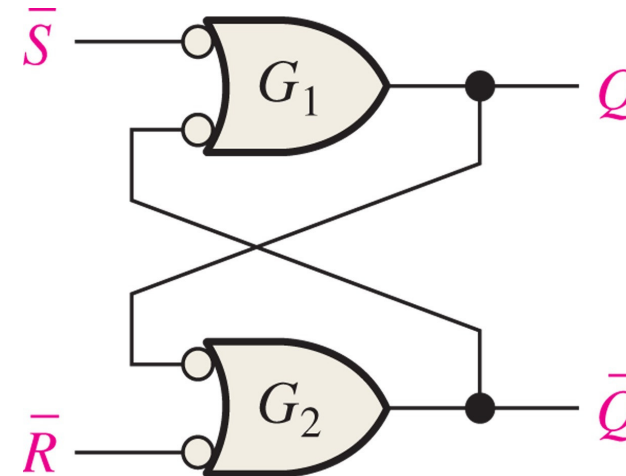


(b) Active-LOW input  
 $\bar{S}$ - $\bar{R}$  latch

# $\bar{S}$ - $\bar{R}$ Latch



(b) Active-LOW input  $\bar{S}$ - $\bar{R}$  latch

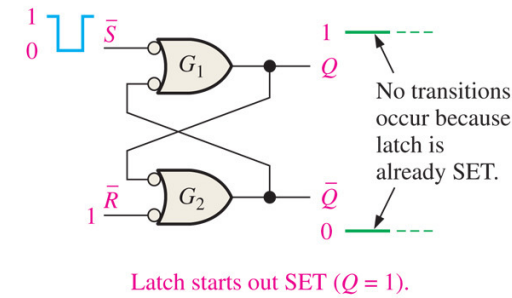
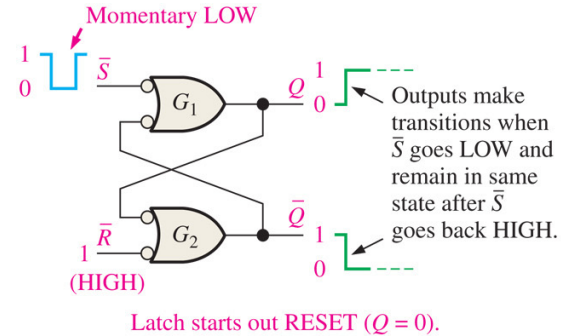


Negative-OR equivalent of the NAND gate S-R latch

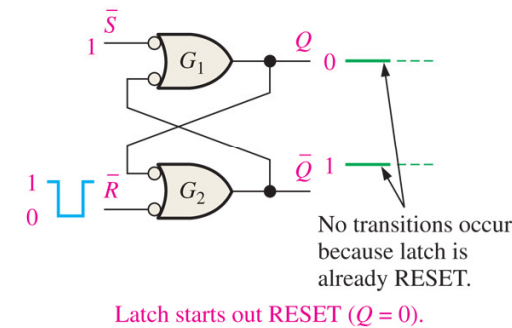
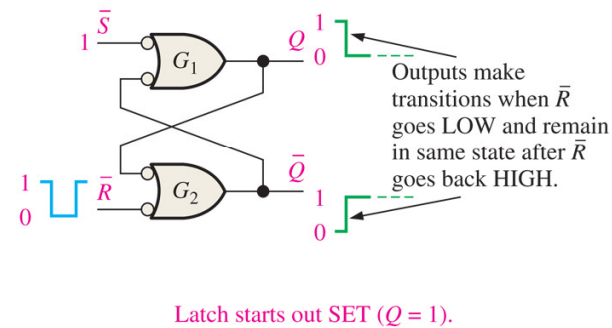
Truth table for an active-LOW input  $\bar{S}$ - $\bar{R}$  latch.

| Inputs    |           | Outputs |           | Comments                                   |
|-----------|-----------|---------|-----------|--------------------------------------------|
| $\bar{S}$ | $\bar{R}$ | $Q$     | $\bar{Q}$ |                                            |
| 1         | 1         | NC      | NC        | No change. Latch remains in present state. |
| 0         | 1         | 1       | 0         | Latch SET.                                 |
| 1         | 0         | 0       | 1         | Latch RESET.                               |
| 0         | 0         | 1       | 1         | Invalid condition                          |

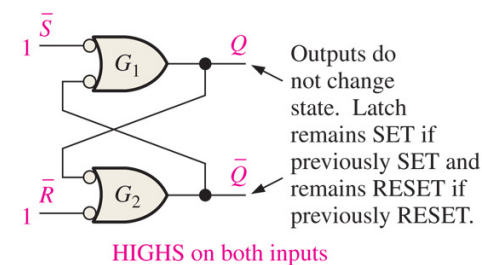
## The three modes of basic $\bar{S}$ - $\bar{R}$ latch operation (SET, RESET, no-change) and the invalid condition



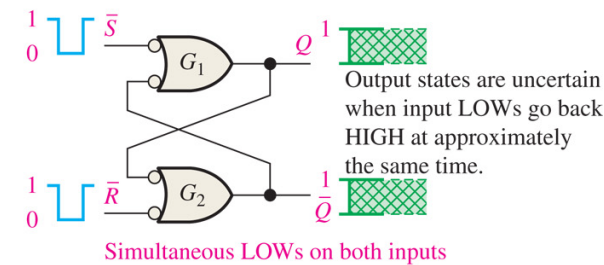
(a) Two possibilities for the SET operation



(b) Two possibilities for the RESET operation

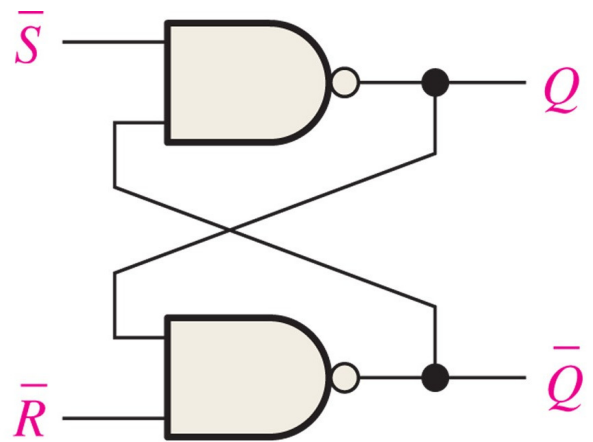


(c) No-change condition



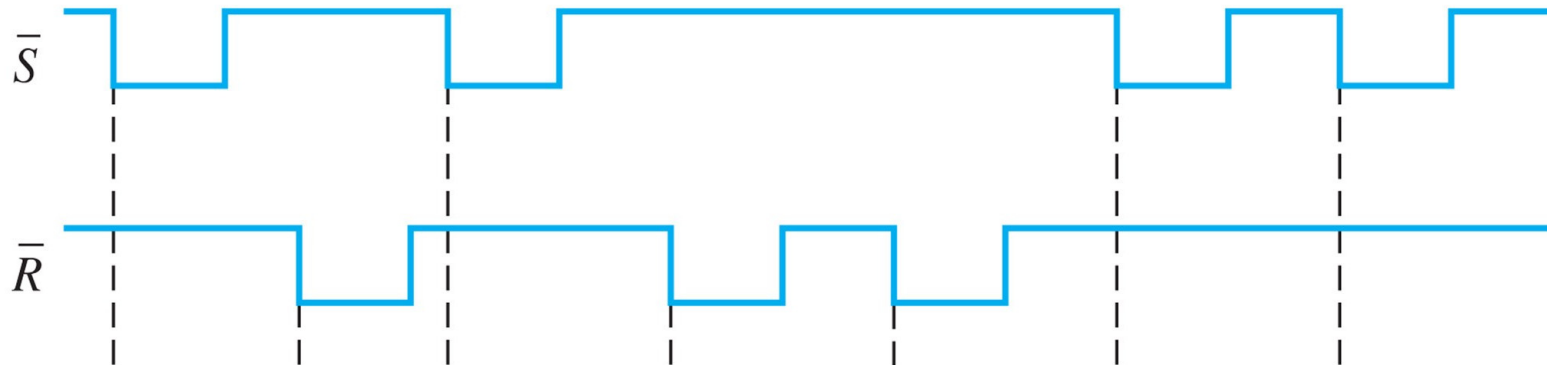
(d) Invalid condition

# $\bar{S}$ - $\bar{R}$ Latch



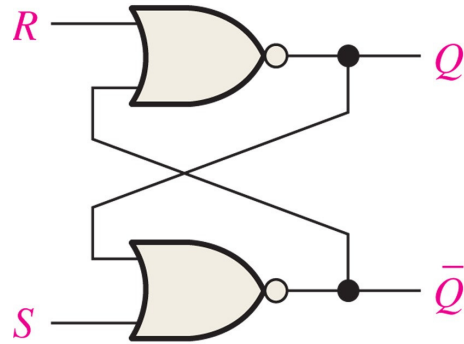
Plot output Q for given inputs below time diagram

(b) Active-LOW input  $\bar{S}$ - $\bar{R}$  latch





Cross-coupled NOR gates



(a) Active-HIGH input S-R latch

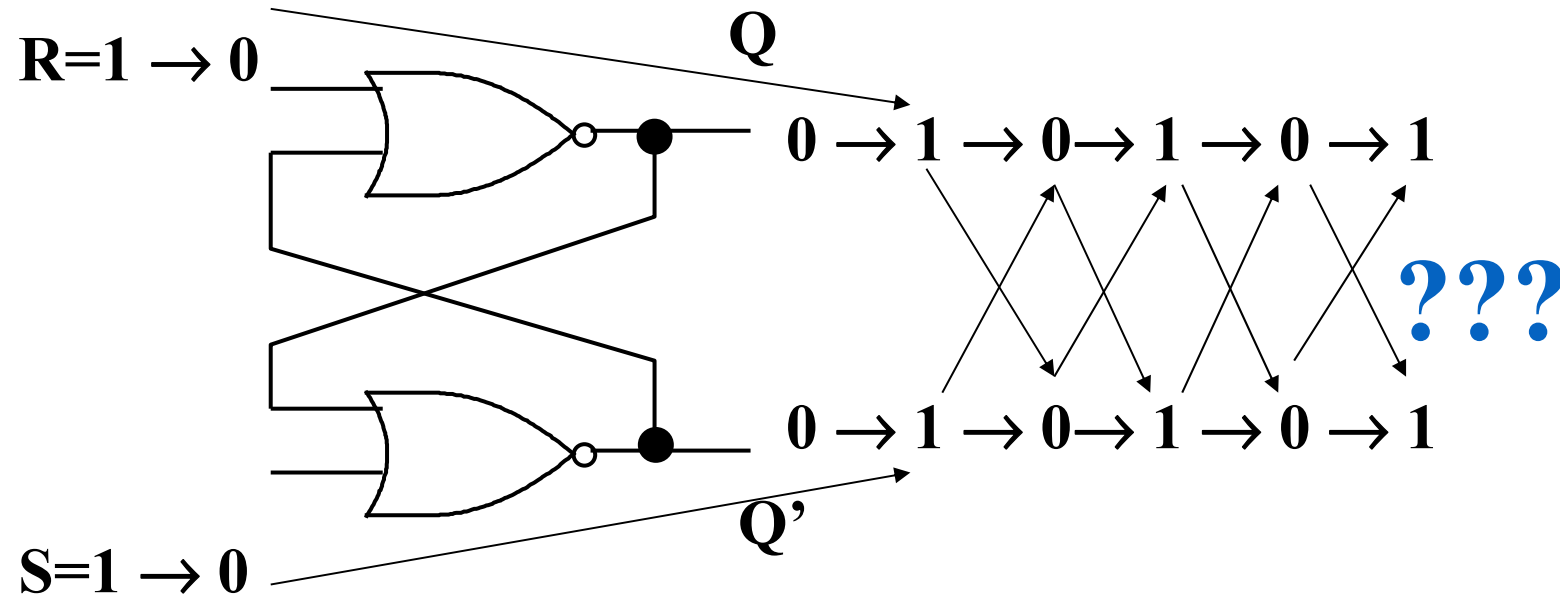
Truth table for an active-HIGH S-R Latch

| R | S | Q | Q' | State              |
|---|---|---|----|--------------------|
| 0 | 0 | Q | Q' | No change (Memory) |
| 0 | 1 | 1 | 0  | Latch Set          |
| 1 | 0 | 0 | 1  | Latch Reset        |
| 1 | 1 | 0 | 0  | Invalid condition  |

Truth table for an active-LOW input  $\bar{S}$ - $\bar{R}$  latch.

| Inputs    |           | Outputs |           | Comments                                   |
|-----------|-----------|---------|-----------|--------------------------------------------|
| $\bar{S}$ | $\bar{R}$ | Q       | $\bar{Q}$ |                                            |
| 1         | 1         | NC      | NC        | No change. Latch remains in present state. |
| 0         | 1         | 1       | 0         | Latch SET.                                 |
| 1         | 0         | 0       | 1         | Latch RESET.                               |
| 0         | 0         | 1       | 1         | Invalid condition                          |

**Invalid Condition:** what happens when S,R return to 0?

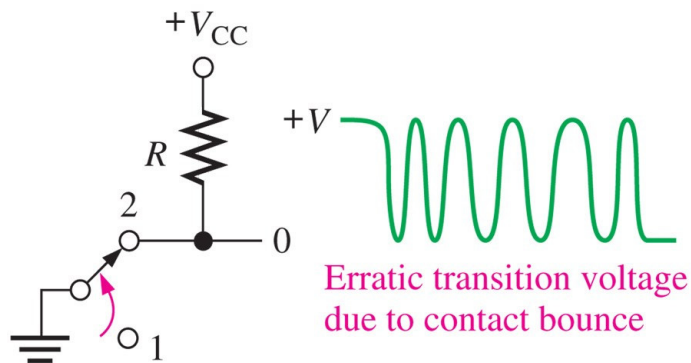


**Undefined next state** when S, R return to 0 simultaneously! At some point the system will settle into a stable condition. **The bottom line is that  $S=R=1$  is an illegal input condition**

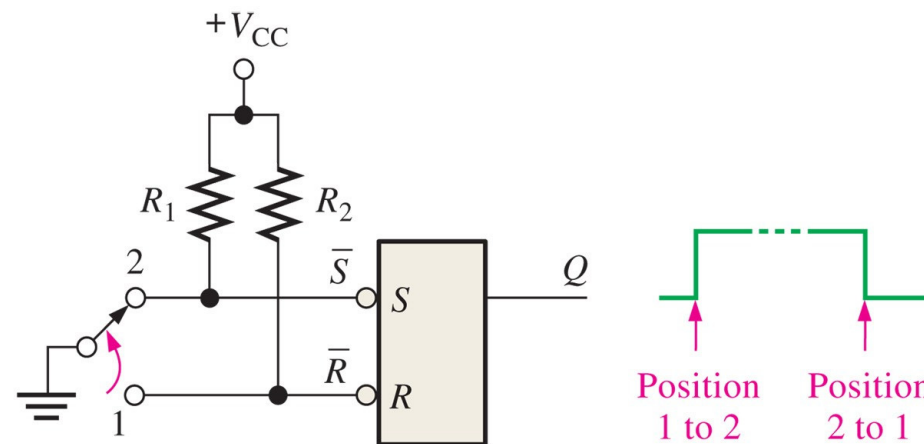
**NB.** The SR latch can be designed such that one input is dominant, i.e. “gates” (disables) the other, e.g.:  **$S = S \text{ AND } (\text{NOT } R)$**

# The Latch as Contact Bounce Eliminator

- Mechanical switch contact “bounce.”
- When the pole of a switch strikes the contact upon switch closure,
  - it physically vibrates or bounces several times before finally making a solid contact.
- Although these bounces are very short in duration, they produce voltage spikes that are often not acceptable in a digital system.
- An S'-R' latch can be used to eliminate the effects of switch bounce
  - Any further voltage spikes on the S' input due to switch bounce do not affect the latch, and it remains SET. Opposite is true for R' (RESET)

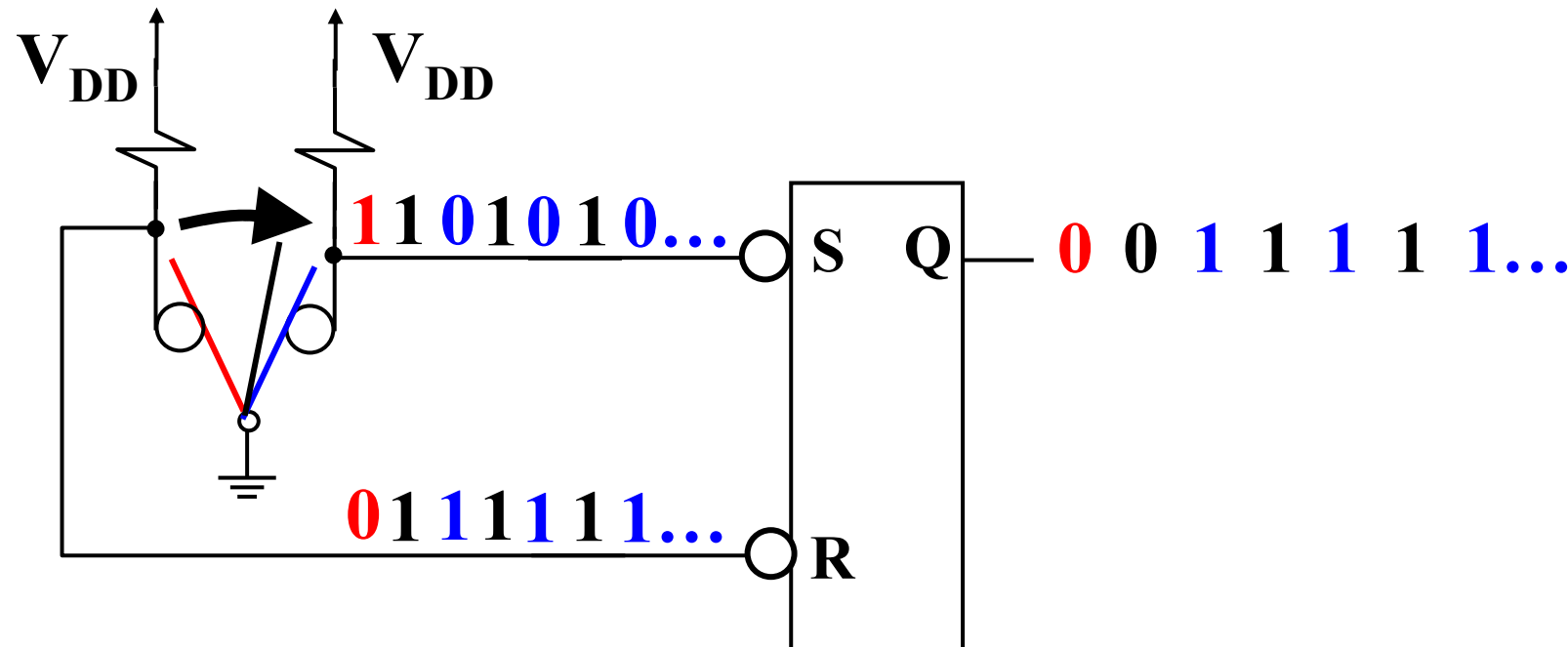


(a) Switch contact bounce



(b) Contact-bounce eliminator circuit

# The Latch as Contact Bounce Eliminator



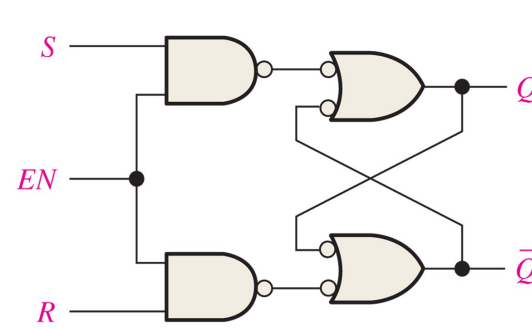
**When switch is in R position, Q is low.**

**When switch is in neither R nor S position, no change in Q .**

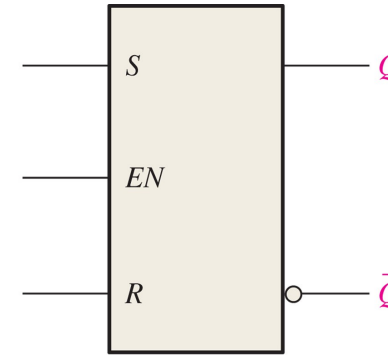
**When switch is in S position, Q is high.**

**When switch is flipped, mechanical bounce occurs on either the S or R terminal. S'-R' Latch prevents bounce from being seen on Q output.**

# Gated S-R Latch

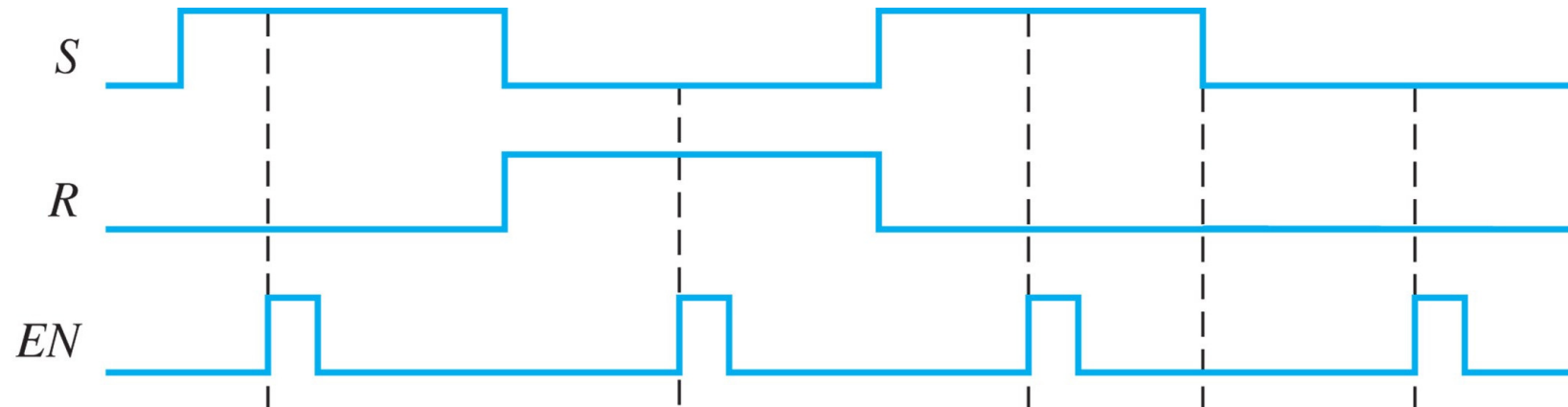


(a) Logic diagram

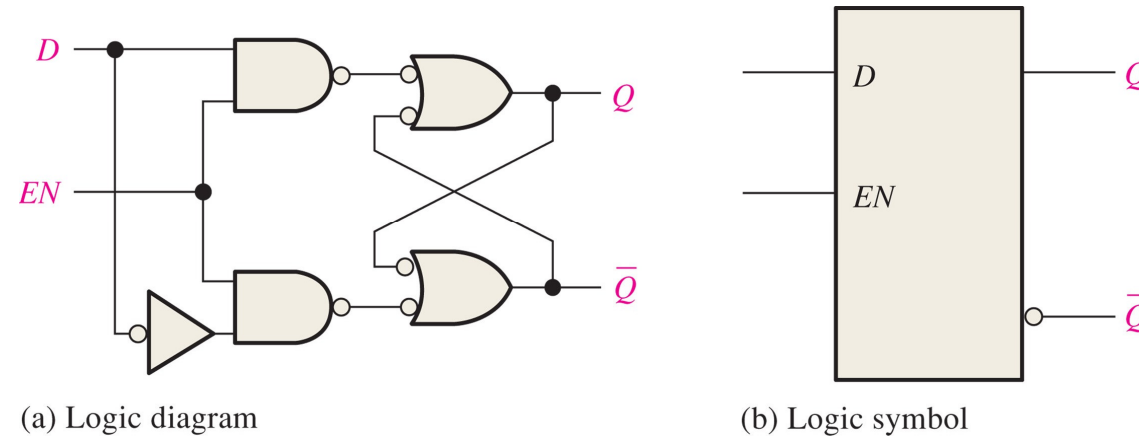


(b) Logic symbol

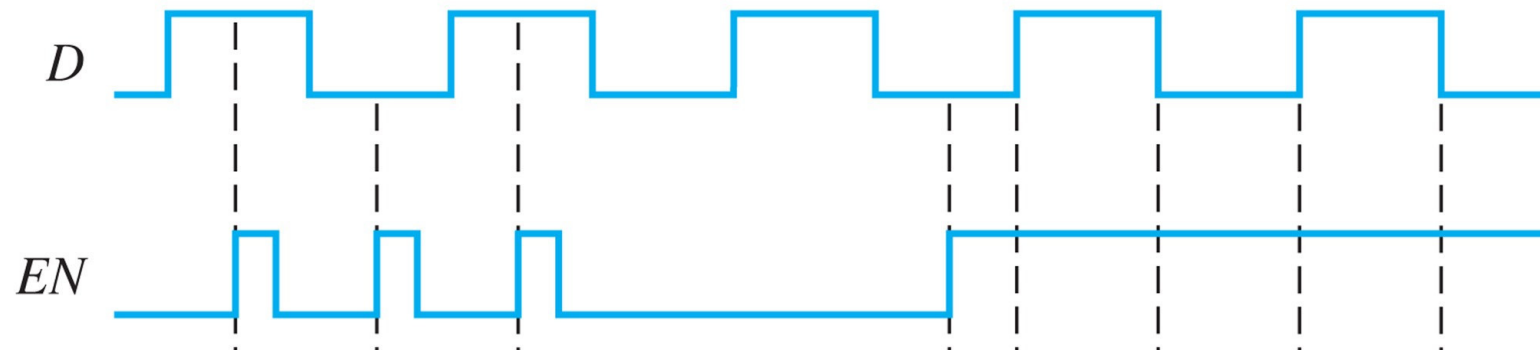
**Plot output  $Q$  for given inputs below time diagram**



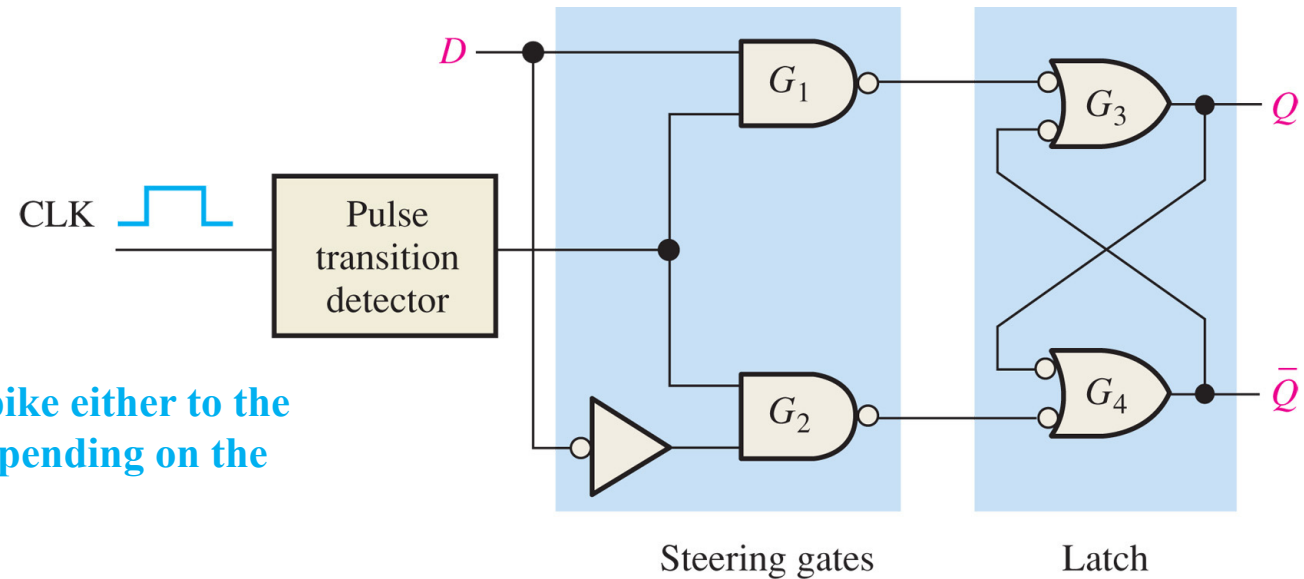
# Gated D Latch



**Plot output  $Q$  for given inputs below time diagram**



Notice that the basic D flip-flop differs from the gated D latch only in that it has a pulse transition detector.

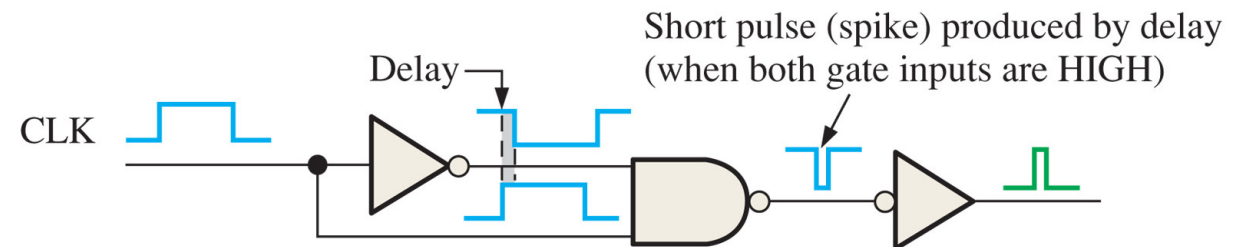


(a) A simplified logic diagram for a positive edge-triggered D flip-flop

The steering gates direct, or steer, the clock spike either to the input to gate  $G_3$  or to the input to gate  $G_4$ , depending on the state of the D input.

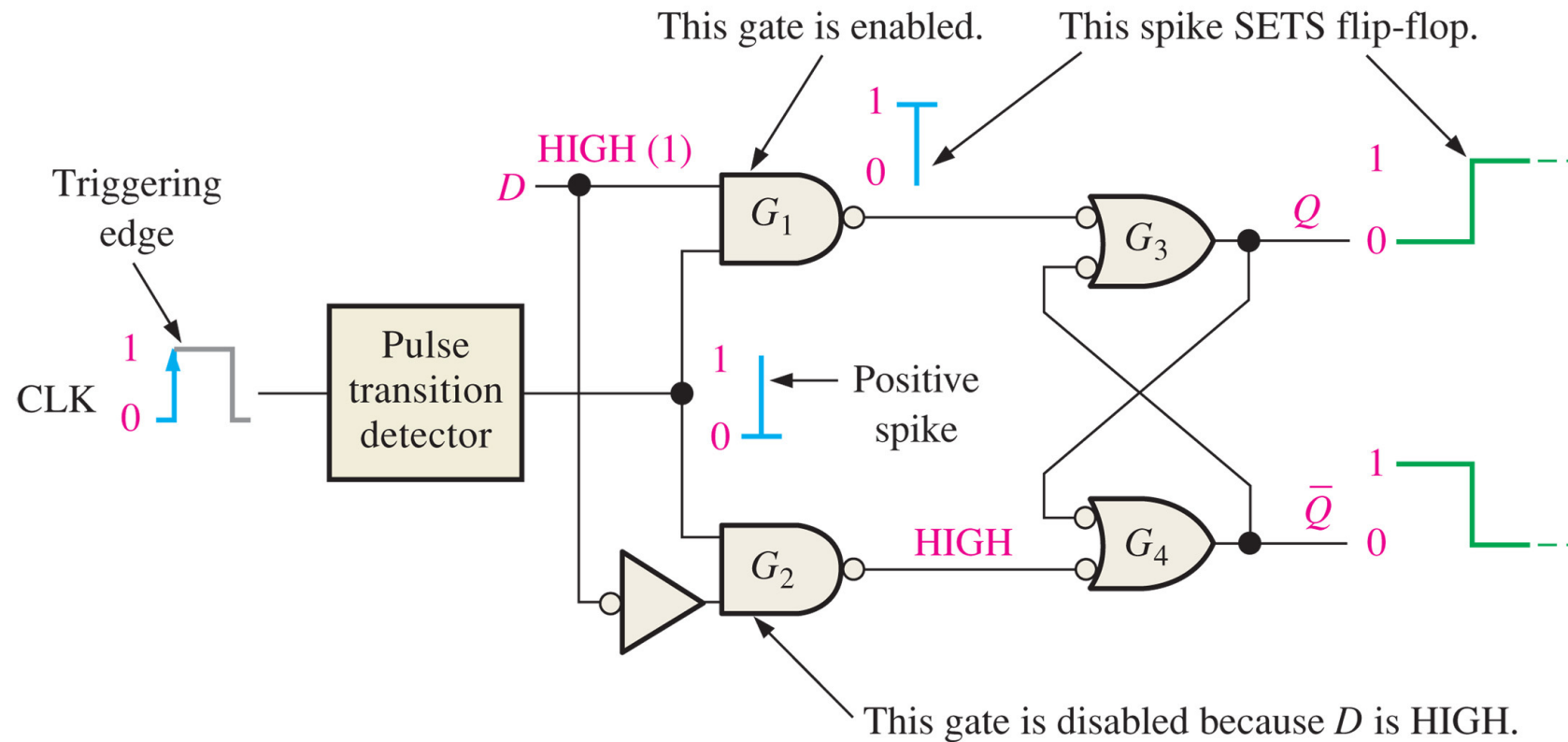
This circuit produces a very short-duration spike on the positive-going transition of the clock pulse.

In a negative edge-triggered flip-flop the clock pulse is inverted first, thus producing a narrow spike on the negative-going edge.



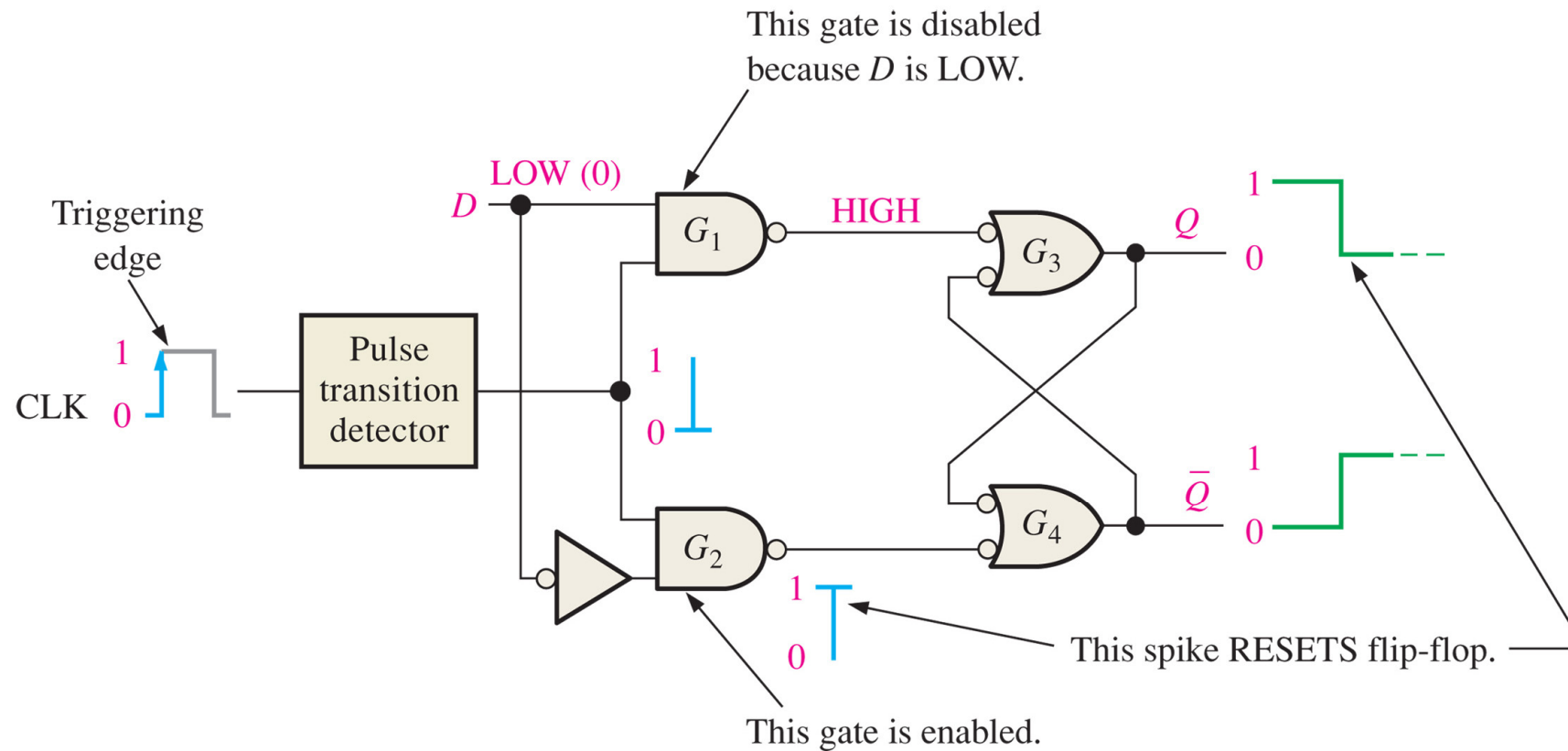
(b) A type of pulse transition detector

Flip-flop making a transition from the RESET state to the SET state on the positive-going edge of the clock pulse





Flip-flop making a transition from the SET state to the RESET state on the positive-going edge of the clock pulse.

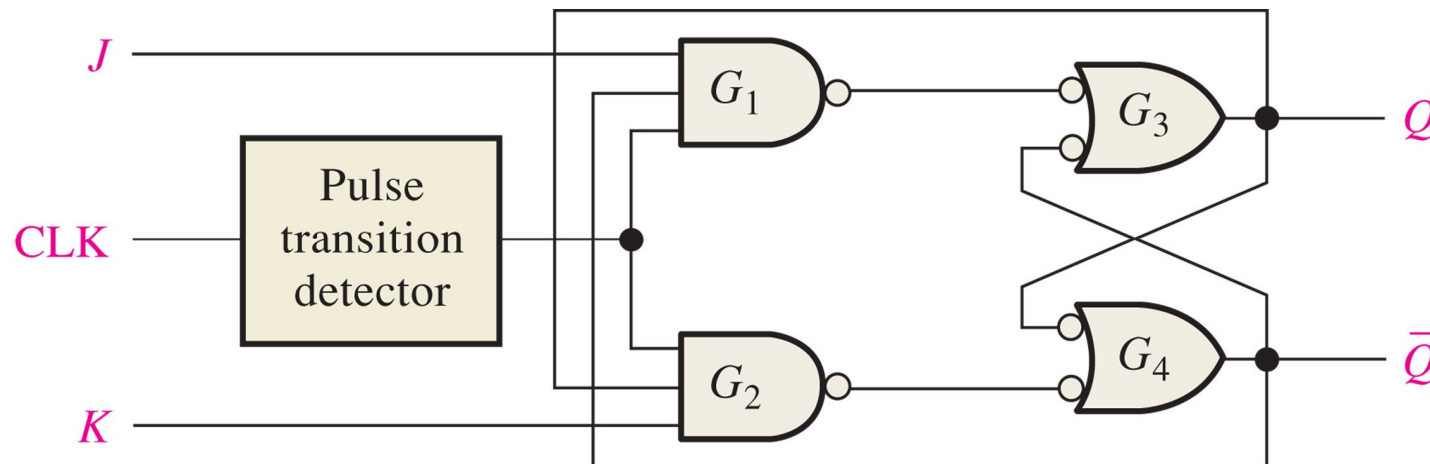
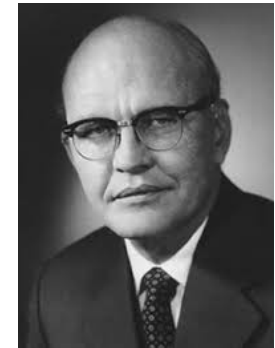


# J-K Flip-flop

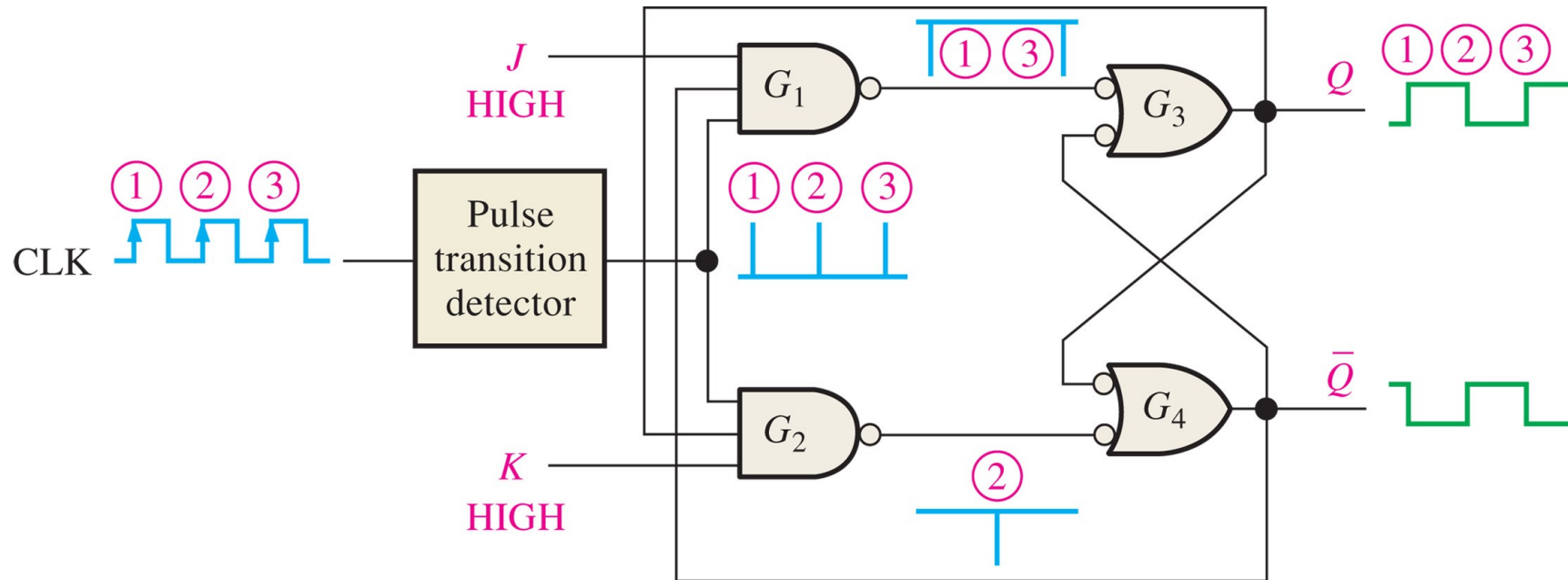
A simplified logic diagram for a positive edge-triggered J-K flip-flop.

The two control inputs are labeled J and K in honor of Jack Kilby, who invented the integrated circuit.

*Jack Kilby invented the integrated circuit while working at Texas Instruments in 1958. He was awarded the Nobel Prize in Physics on December 10, 2000*



Transitions illustrating flip-flop operation.





THE HONG KONG  
UNIVERSITY OF SCIENCE  
AND TECHNOLOGY

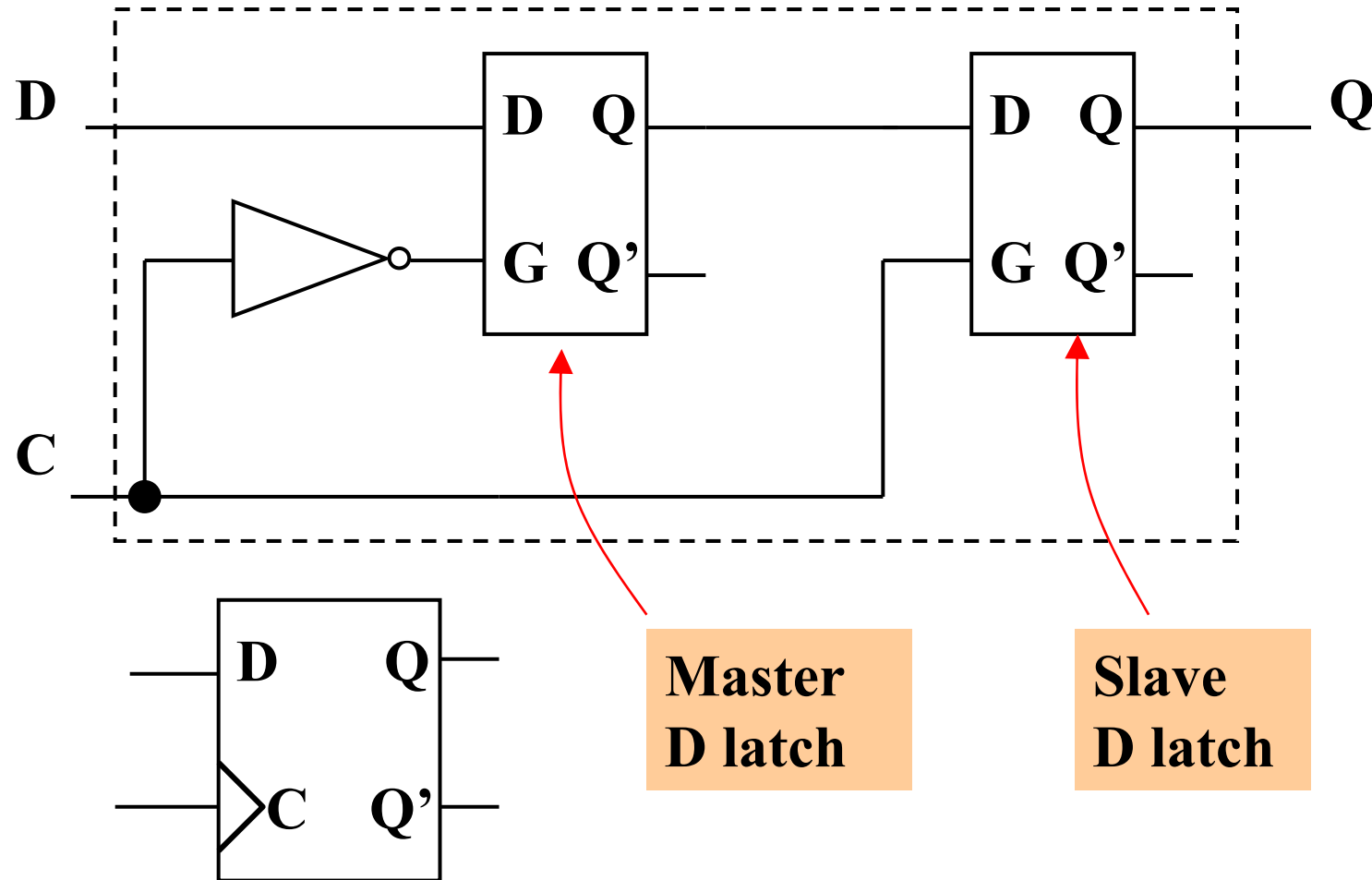
## Master/Slave arrangement of two D-type latches

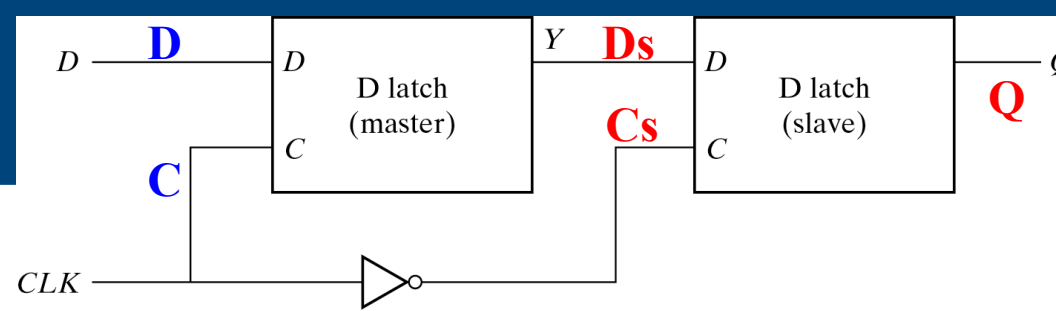


# DFF Master/Slave Implementation (rising edge triggered)



THE HONG KONG  
UNIVERSITY OF SCIENCE  
AND TECHNOLOGY





**D: master**

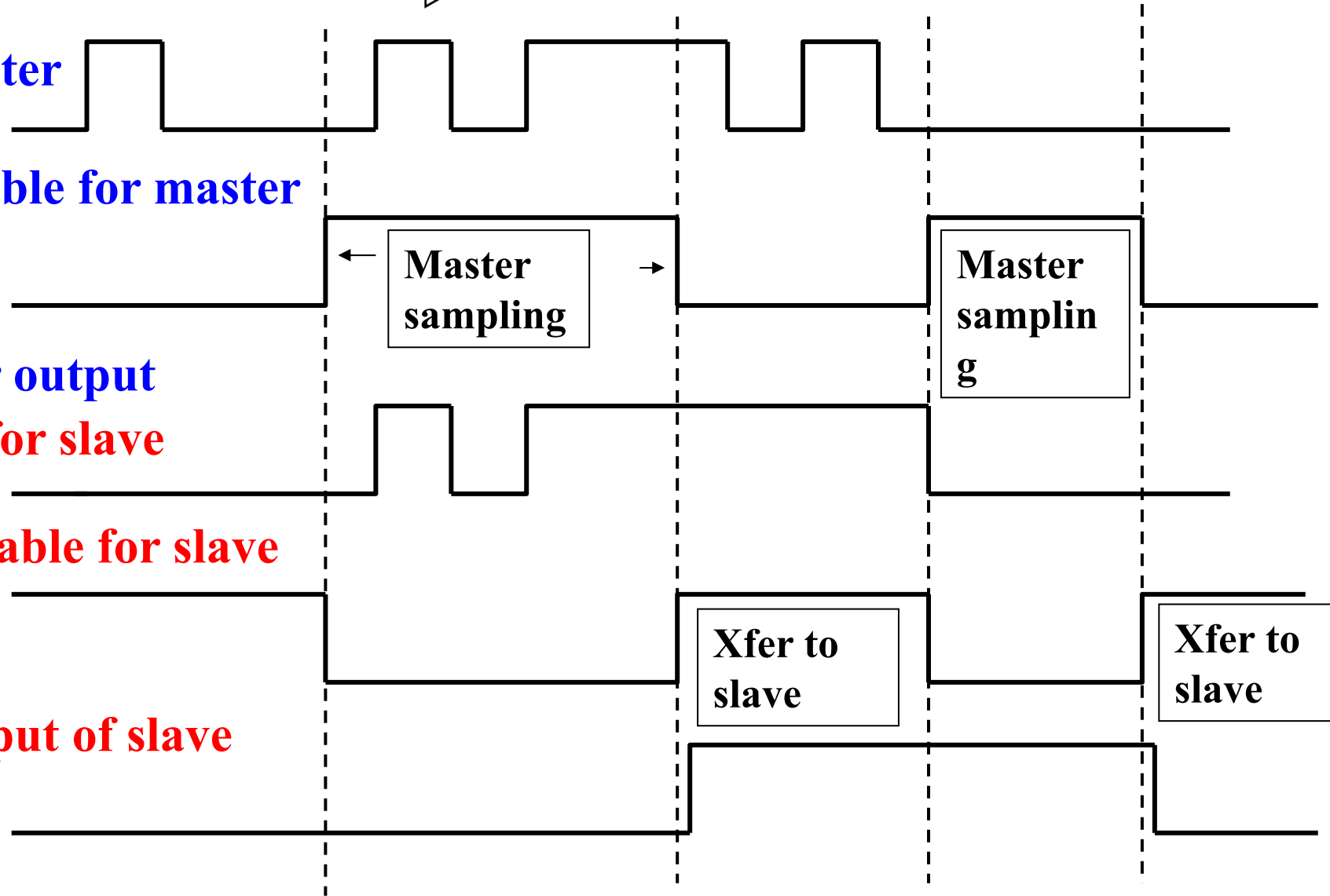
**C: Enable for master**

**Master output**

**Ds: D for slave**

**Cs: Enable for slave**

**Q: output of slave**



# D Flip-flop Implementation with RS latches

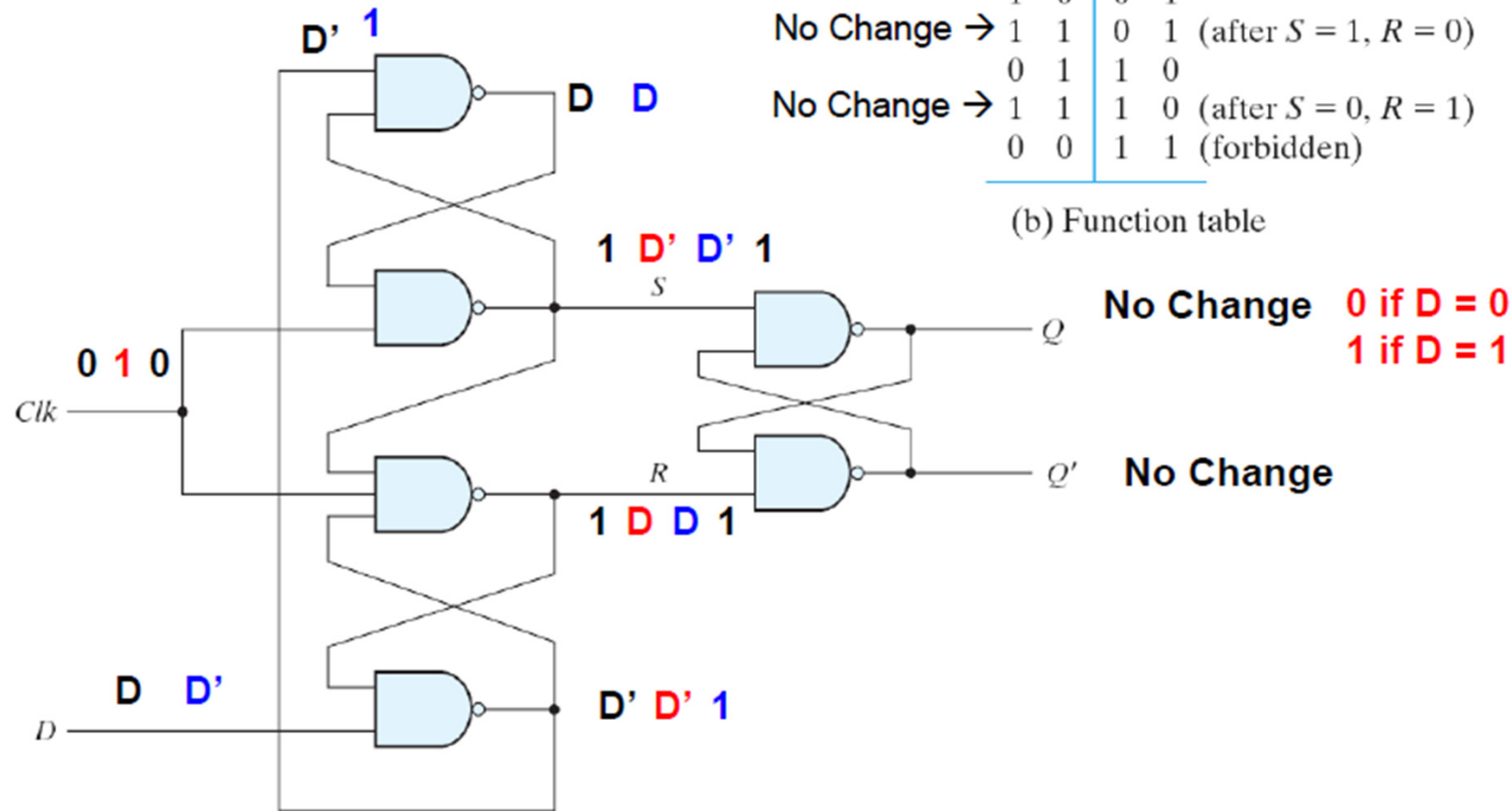


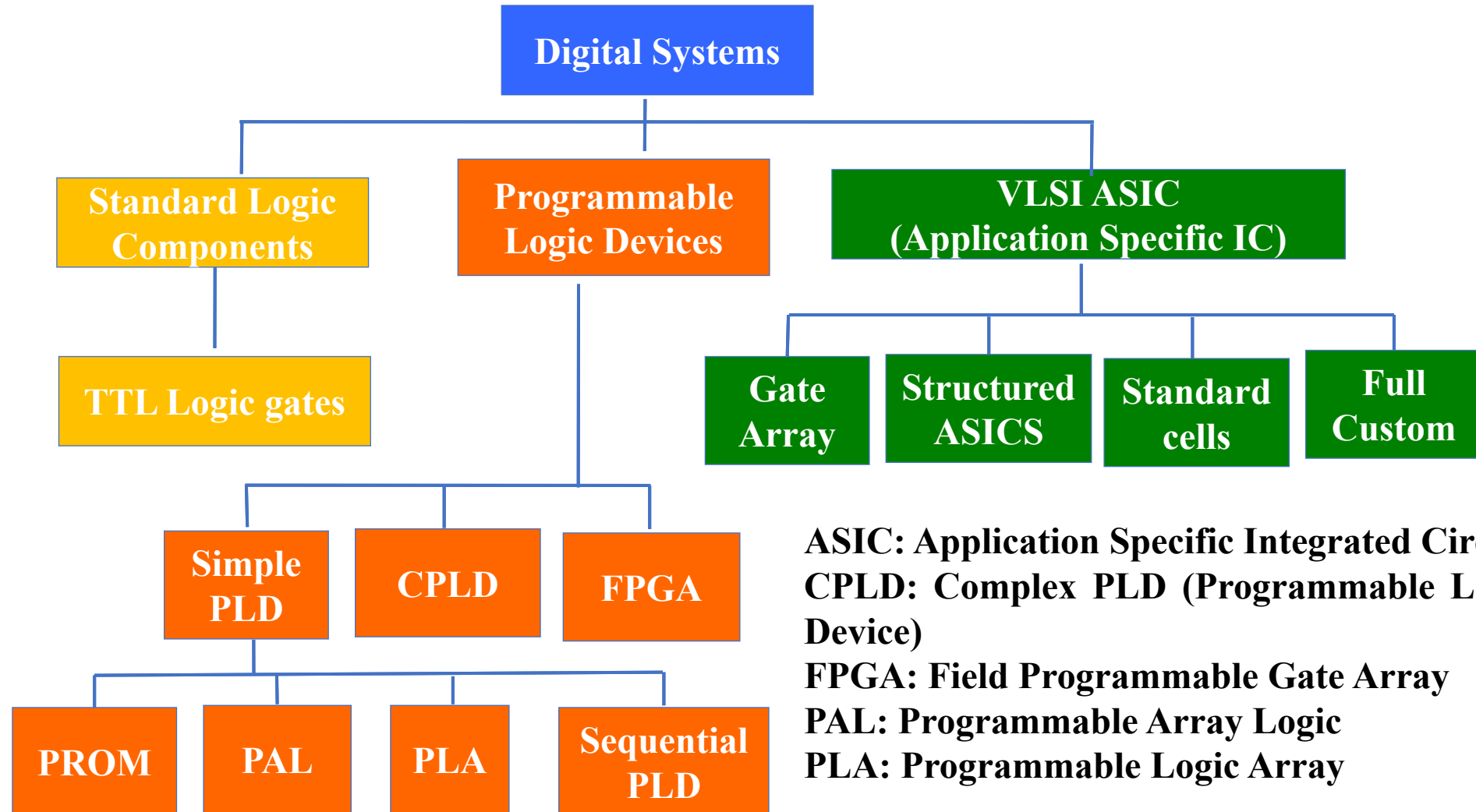
When CLK = 0, Change in D does NOT change Q

When CLK = 1, Change in D does NOT change Q

| S | R | Q | Q' |
|---|---|---|----|
| 1 | 0 | 0 | 1  |
| 1 | 1 | 0 | 1  |
| 0 | 1 | 1 | 0  |
| 1 | 1 | 1 | 0  |
| 0 | 0 | 1 | 1  |

(b) Function table







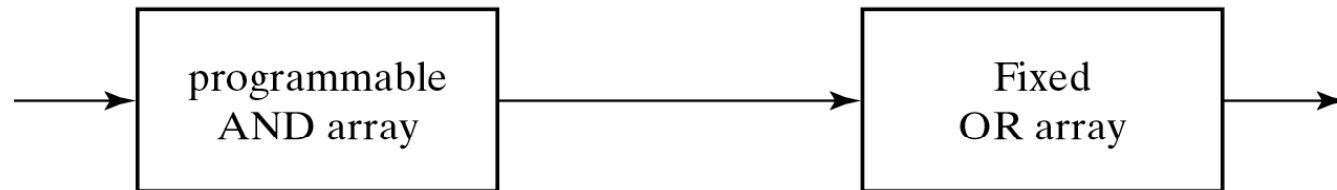
# Simple PLDs Three Combinational PLDs



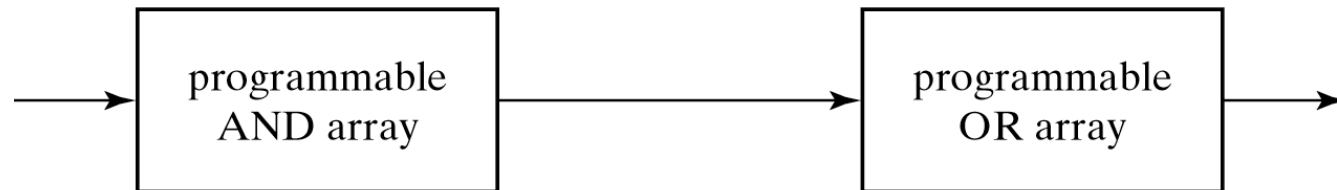
THE HONG KONG  
UNIVERSITY OF SCIENCE  
AND TECHNOLOGY



(a) Programmable read-only memory (PROM)



(b) Programmable array logic (PAL)



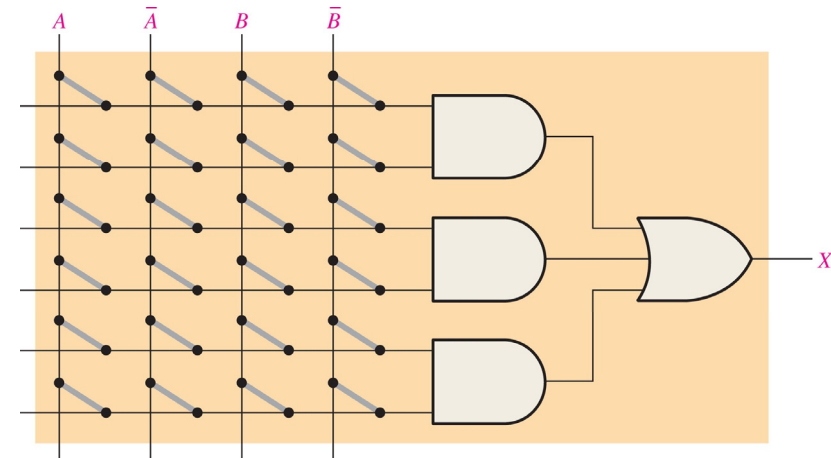
(c) Programmable logic array (PLA)

# Programmable Array Logic (PAL)



- A programmable array is a grid or matrix of conductors that form rows and columns with a programmable link at each cross point.
- Each programmable link, which is a fuse in the case of a PAL, is called a cell.
- Each row is connected to the input of an AND gate, and each column is connected to an input variable or its complement.
- By programming the presence or absence of a fuse connection, any combination of input variables or complements can be applied to an AND gate to form any desired product term.
- The AND gates are connected to an OR gate, creating a sum-of-products (SOP) output.

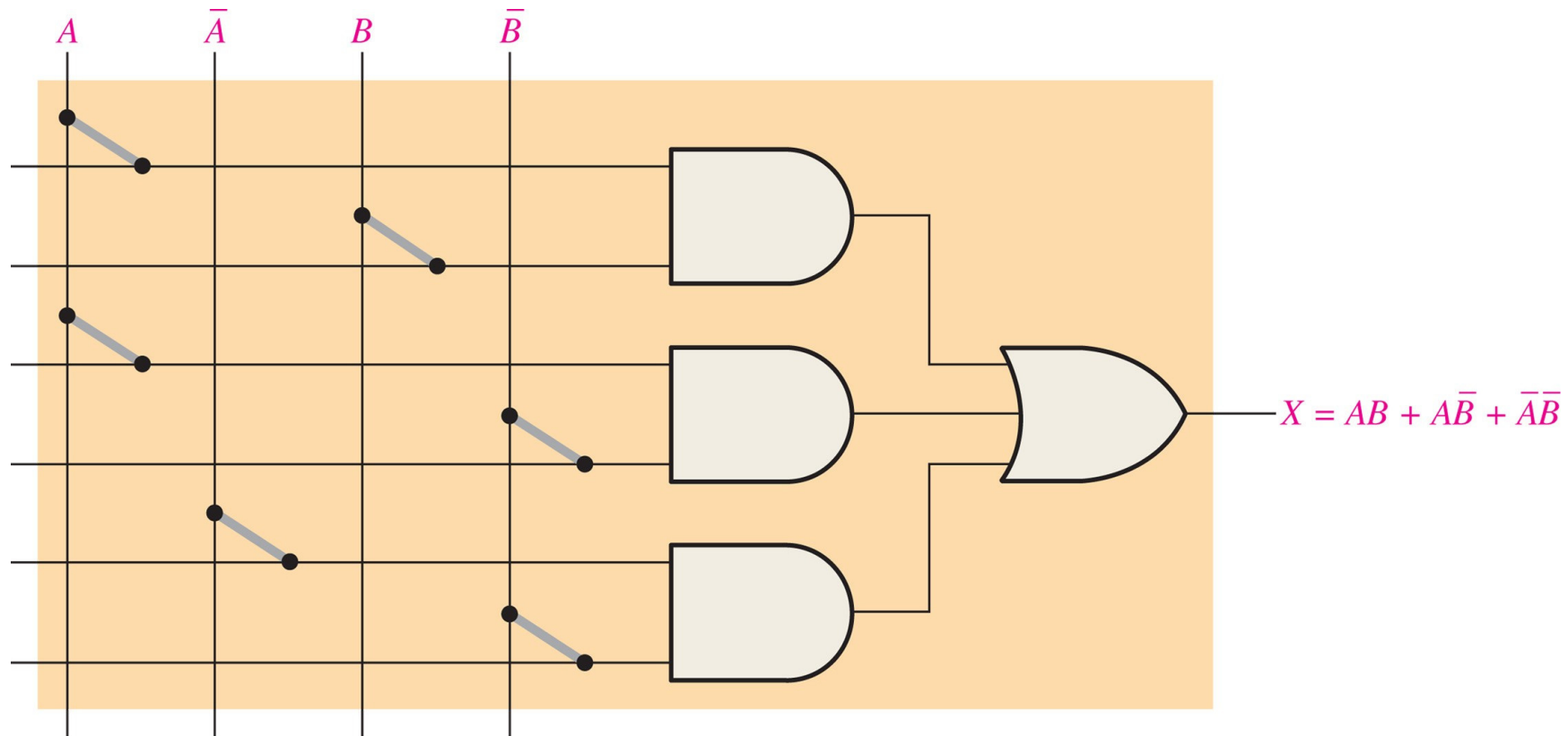
**Basic AND/OR structure of a PAL.**



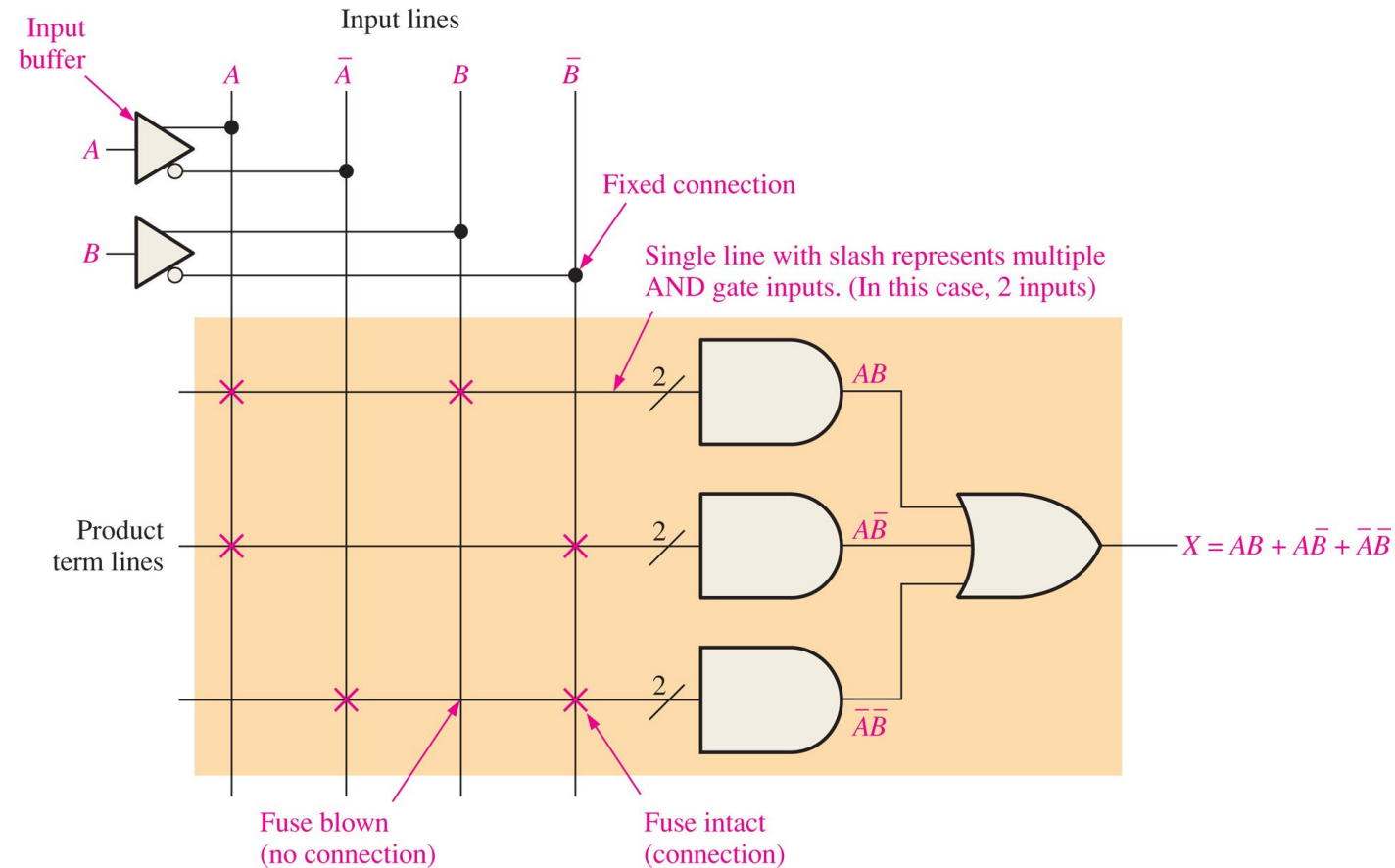
Generally, PALs are implemented with fuse process technology and are, therefore, one-time programmable (OTP)

### PAL implementation of a SOP expression

the fuses are left intact to connect the desired variables or their complements to the appropriate AND gate inputs

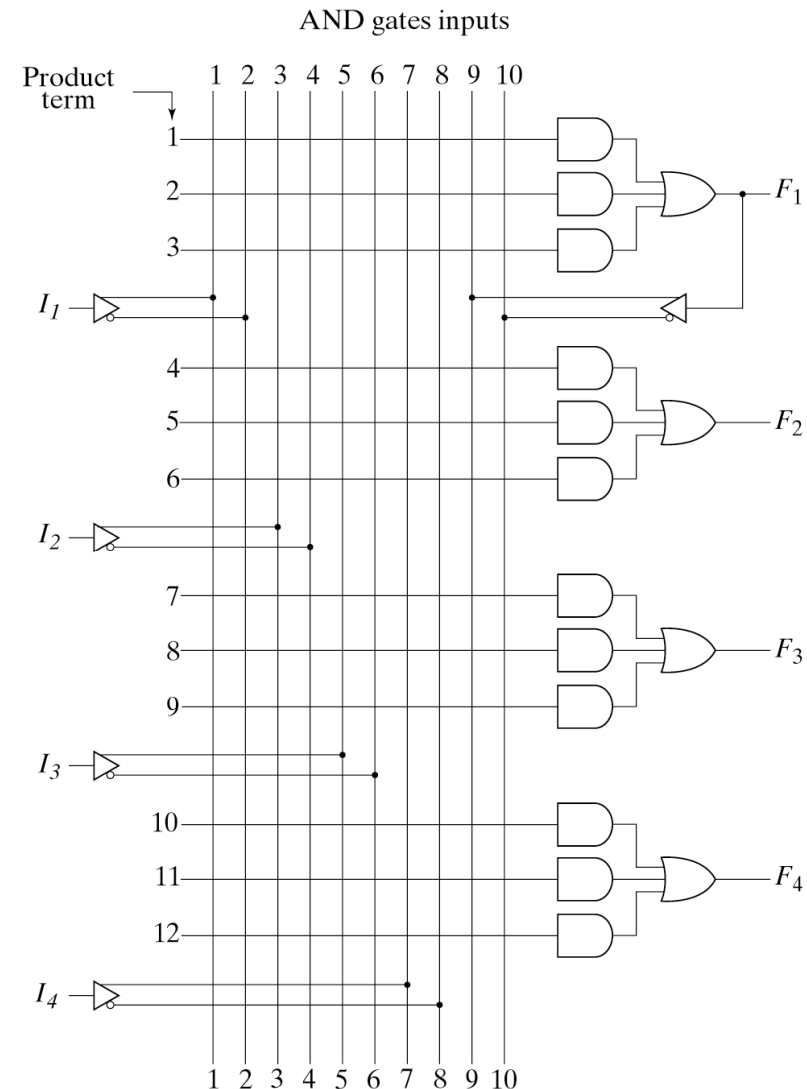


## A portion of a programmed PAL



# PAL Example: 4 Inputs, 12 AND gates and 4 Outputs

- **4 external inputs**
  - 8 inverted/normal inputs to AND gate through inverters
- **12 AND gates**
  - Divided into 4 groups for 4 OR gates
  - Each group has 3 AND gates
- **10-input AND gates**
  - 8 are from external inputs
  - 2 are from output feedback
    - » What is this for?
- **4 3-input OR gates**
  - for 4 outputs
  - with at most 3 product terms



# Design with PAL, example

- Given functions

$$w = ABC' + A'B'CD';$$

$$x = A + BCD;$$

$$y = A'B + CD + B'D';$$

$$z = (ABC' + A'B'CD') + AC'D' + A'B'C'D$$

- $w$  and  $x$  has 2 product terms,  $y$  has 3 and  $z$  has 4
- But we have to use up all the 3 product terms since they are fixed

- Re-express

$$w = ABC' + A'B'CD' + \mathbf{0};$$

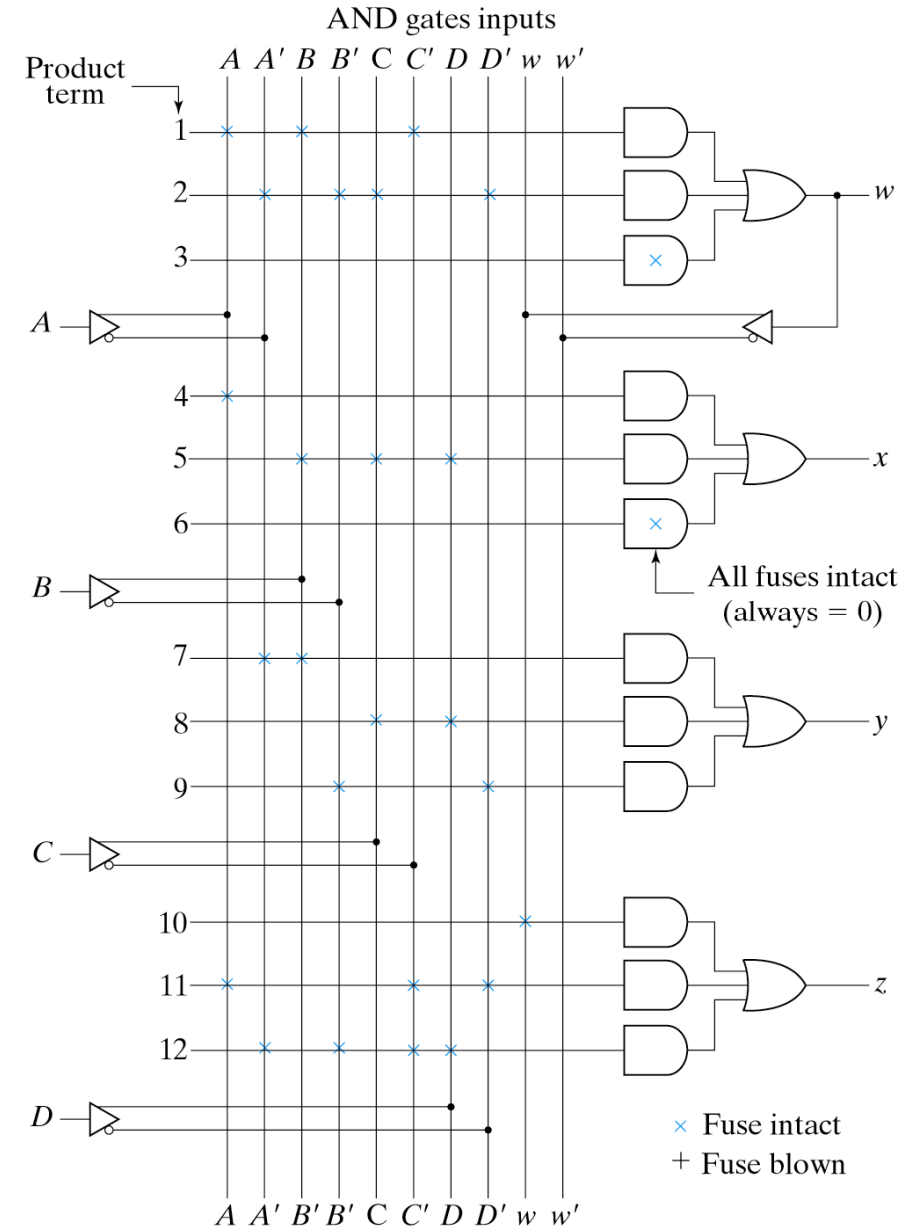
$$x = A + BCD + \mathbf{0};$$

$$y = A'B + CD + B'D';$$

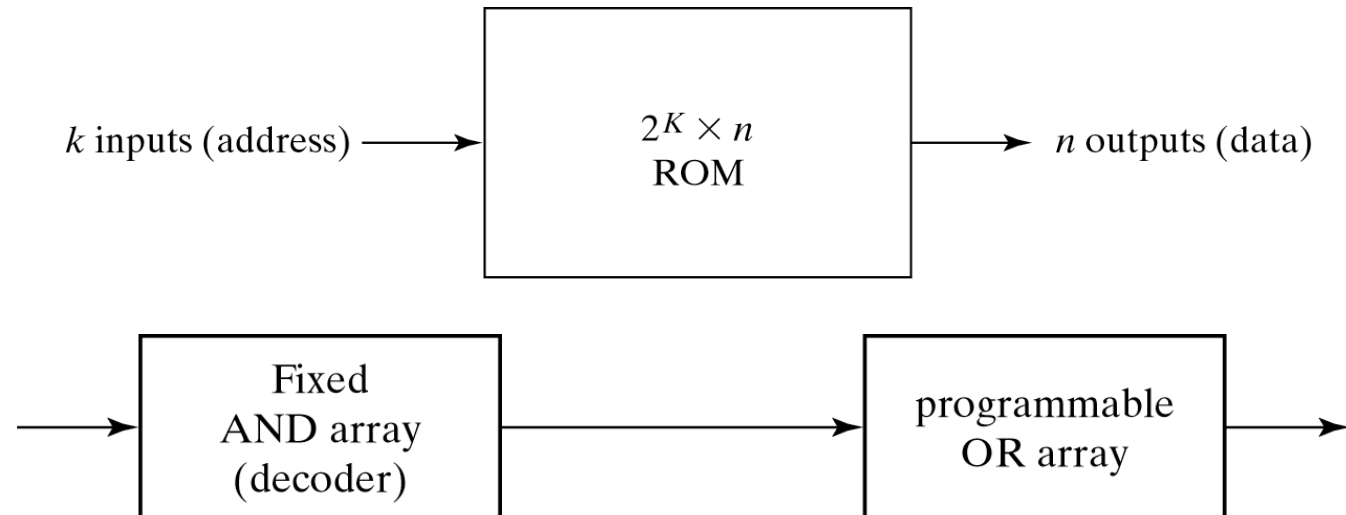
$$z = \mathbf{w} + AC'D' + A'B'C'D$$

» Note that  $AA'BB'CC'DD'ww' = 0$

- Output feedback allows for more product terms



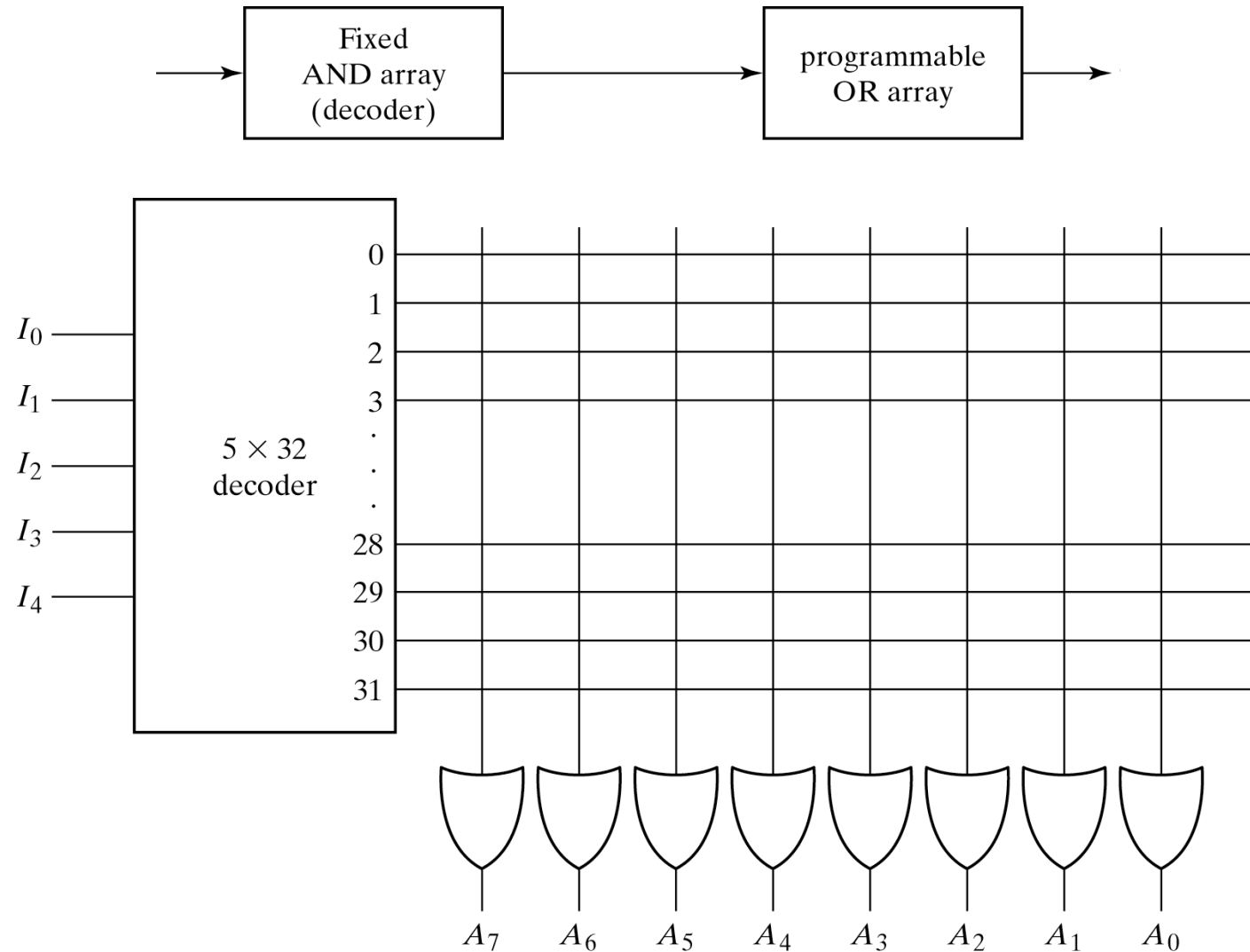
- **Decoder provides all the minterms (AND of inputs)**
  - $k$  inputs for  $2^k$  minterms
  - All the minterms are available for every OR gate
- **OR gate selects the minterms for outputs**
  - OR array is programmable
  - $n$  number of OR gates for  $n$  outputs
- **Each OR gate receives minterms according to its SOP**



# Programmable Read-Only Memory (PROM)



THE HONG KONG  
UNIVERSITY OF SCIENCE  
AND TECHNOLOGY

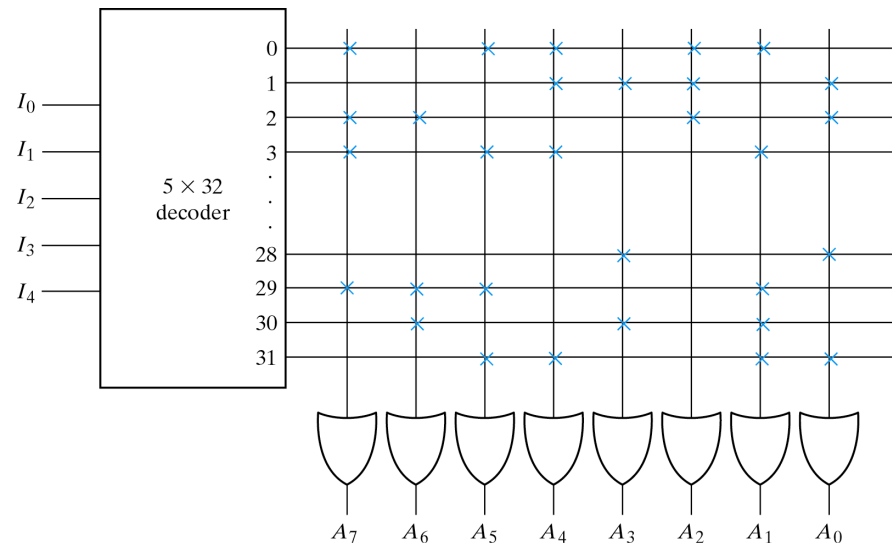




# Use of PROM for Combinational Logic Design

- Given project specifications
  - 5 inputs, 8 outputs
- Derive the truth table
- K-map ?
  - No minimization

| Inputs |    |    |    |    | Outputs |    |    |    |    |    |    |    |
|--------|----|----|----|----|---------|----|----|----|----|----|----|----|
| I4     | I3 | I2 | I1 | I0 | A7      | A6 | A5 | A4 | A3 | A2 | A1 | A0 |
| 0      | 0  | 0  | 0  | 0  | 1       | 0  | 1  | 1  | 0  | 1  | 1  | 0  |
| 0      | 0  | 0  | 0  | 1  | 0       | 0  | 0  | 1  | 1  | 1  | 0  | 1  |
| 0      | 0  | 0  | 1  | 0  | 1       | 1  | 0  | 0  | 0  | 1  | 0  | 1  |
| 0      | 0  | 0  | 1  | 1  | 1       | 0  | 1  | 1  | 0  | 0  | 1  | 0  |
| 1      | 1  | 1  | 0  | 0  | 0       | 0  | 0  | 0  | 1  | 0  | 0  | 1  |
| 1      | 1  | 1  | 0  | 1  | 1       | 1  | 1  | 0  | 0  | 0  | 1  | 0  |
| 1      | 1  | 1  | 1  | 0  | 0       | 1  | 0  | 0  | 1  | 0  | 1  | 0  |
| 1      | 1  | 1  | 1  | 1  | 0       | 0  | 1  | 1  | 0  | 0  | 1  | 1  |

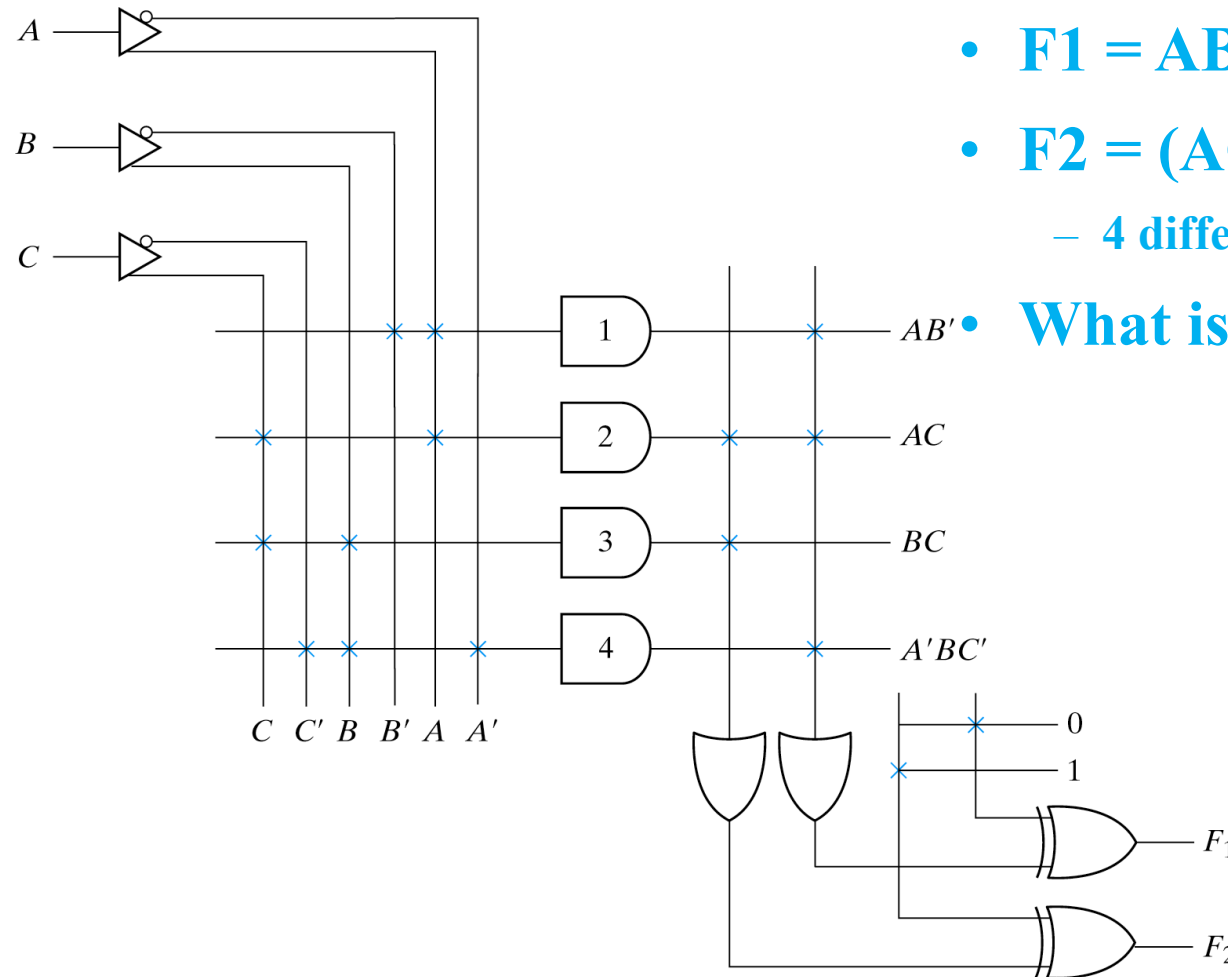


- **Programmable AND - Programmable OR**
- **AND gates produces product terms (*not minterms*)**
  - Number of product terms is determined by number of AND gates
  - Numbers of inputs and AND gates are independent
  - All the product terms are available for every OR gate
- **OR gate selects the product terms for outputs**
  - Different OR gate has different product terms for different functions
  - OR array is programmable
  - $n$  OR gates for  $n$  outputs
  - Each OR gate receives product terms according to its SOP



# PLA Example:

## 3 Inputs, 4 AND gates and 2 Outputs



- $F1 = AB' + AC + A'BC'$

- $F2 = (AC + BC)'$

- 4 different product terms

- What is the XOR gate for?

- **Derive truth table**
- **Logic Optimization**
  - **PLA has limited number of AND gates**
    - » **total number of AND gate or product terms is the bottleneck**
  - **Each AND gate for one product term of any number of variables**
  - **For multiple output functions**
    - » **optimize the total number of different product terms**
    - » **find common product terms to be shared among the outputs**
    - » **K-map is helpful**
- **Select a PLA and find a programmer that supports the PLA**
- **Enter the programming map into the programmer**
- **Download the map to the PLA by the programmer**

- **Given project specifications**
  - $F_1(A, B, C) = \Sigma(0, 1, 2, 4)$ ;  $F_2(A, B, C) = \Sigma(0, 5, 6, 7)$
  - 7 different minterms (0, 1, 2, 4, 5, 6, 7)
- **Derive the truth table or express in K-map**
- **Logic Optimization for the total number of product terms**
  - Conventional optimization for each function
    - » 3 product terms for both  $F_1$  and  $F_2$ , and 6 different product terms in total

|     |   | $BC$ |    | $B$ |    |
|-----|---|------|----|-----|----|
|     |   | 00   | 01 | 11  | 10 |
| $A$ | 0 | 1    | 1  | 0   | 1  |
|     | 1 | 1    | 0  | 0   | 0  |
|     |   |      |    | $C$ |    |

$$F_1 = A'B' + A'C' + B'C'$$

$$F_1 = (AB + AC + BC)'$$

|     |   | $BC$ |    | $B$ |    |
|-----|---|------|----|-----|----|
|     |   | 00   | 01 | 11  | 10 |
| $A$ | 0 | 1    | 0  | 0   | 0  |
|     | 1 | 0    | 1  | 1   | 1  |
|     |   |      |    | $C$ |    |

$$F_2 = AB + AC + A'B'C'$$

$$F_2 = (A'C + A'B + AB'C')'$$

- **Logic Optimization for the total number of product terms**
  - find common product terms to be shared among the outputs
    - » both share  $A'B'C'$ ; still need 6 different product terms in total
  - with XOR gate for output inversion
    - » find common product terms between  $(F_1, F_2)$ ,  $(F_1', F_2)$ ,  $(F_1, F_2')$  and  $(F_1', F_2')$
    - »  $(F_1', F_2)$  has 4 different product terms
  - XOR gates help logic optimization

|     |   |      |    |     |    |
|-----|---|------|----|-----|----|
|     |   | $BC$ |    | $B$ |    |
|     |   | 00   | 01 | 11  | 10 |
| $A$ | 0 | 1    | 1  | 0   | 1  |
| $A$ | 1 | 1    | 0  | 0   | 0  |
|     |   | $C$  |    |     |    |

$$F_1 = A'B' + A'C' + B'C'$$

$$F_1' = (AB + AC + BC)'$$

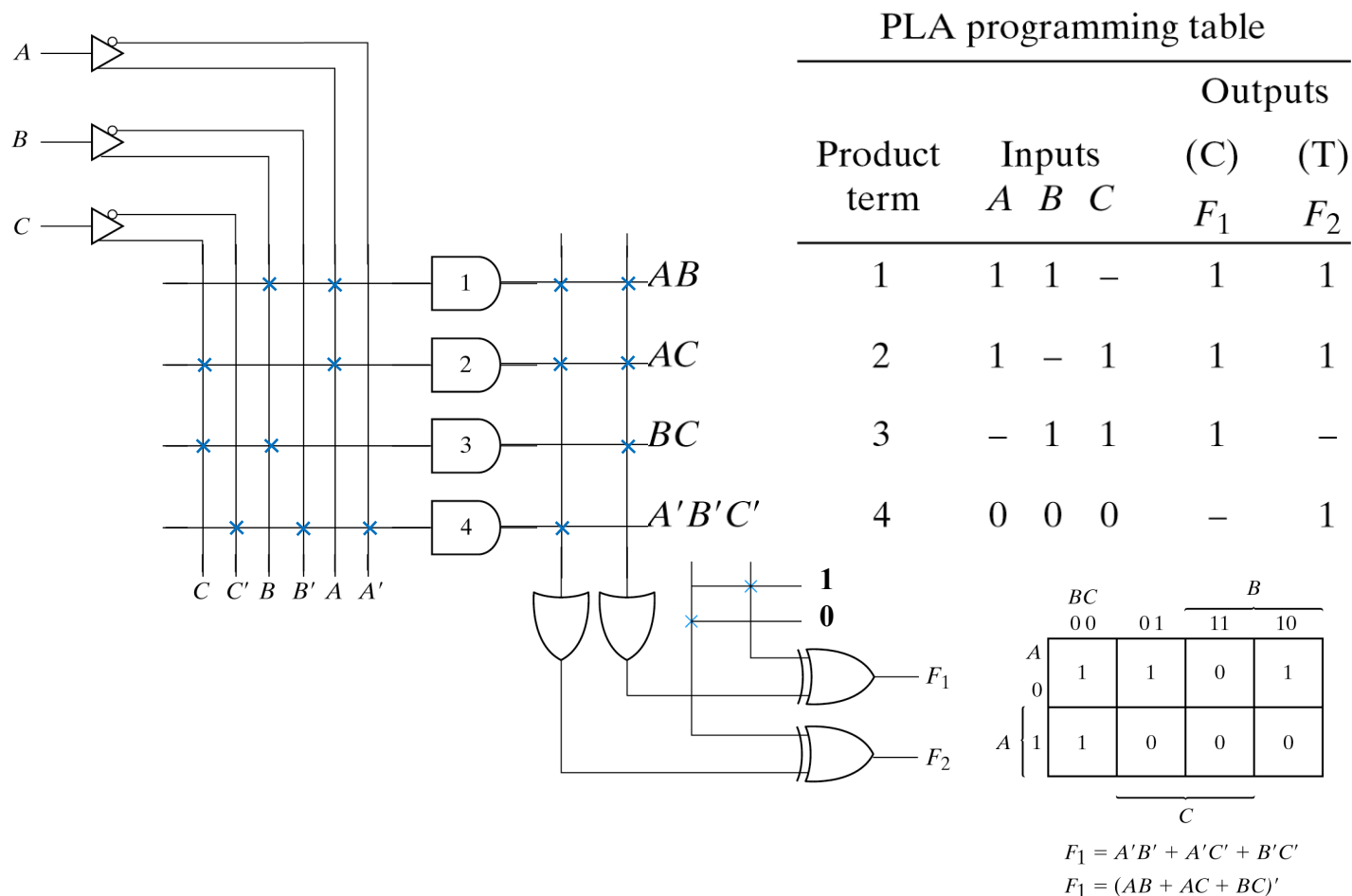
|     |   |      |    |     |    |
|-----|---|------|----|-----|----|
|     |   | $BC$ |    | $B$ |    |
|     |   | 00   | 01 | 11  | 10 |
| $A$ | 0 | 1    | 0  | 0   | 0  |
| $A$ | 1 | 0    | 1  | 1   | 1  |
|     |   | $C$  |    |     |    |

$$F_2 = AB + AC + A'B'C'$$

$$F_2' = (A'C + A'B + AB'C')'$$

# Design with PLA, Implementation

- The fuse map of a PLA can be specified in a tabular form.



*The PLA programming table consists of 3 sections:*

*The first section lists the product terms numerically.*

*The second section specifies the required paths between inputs and AND gates.*

*The third section specifies the paths between the AND and OR gates.*

*For each output variable, we may have a T (for true) or C (for complement) for programming the XOR gate.*

|   |   |    |    |    |    |
|---|---|----|----|----|----|
|   |   | BC |    | B  |    |
|   |   | 00 | 01 | 11 | 10 |
| A | 0 | 1  | 1  | 0  | 1  |
|   | 1 | 1  | 0  | 0  | 0  |

C

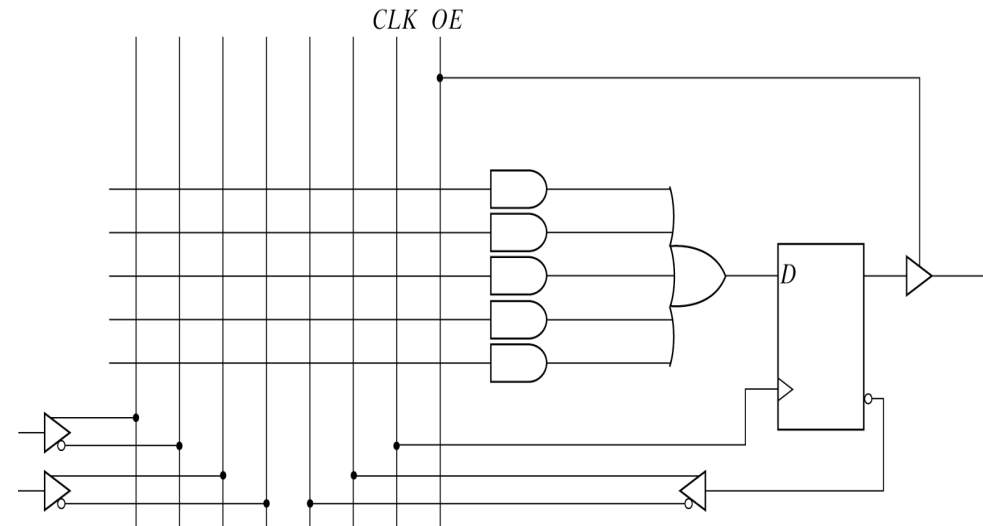
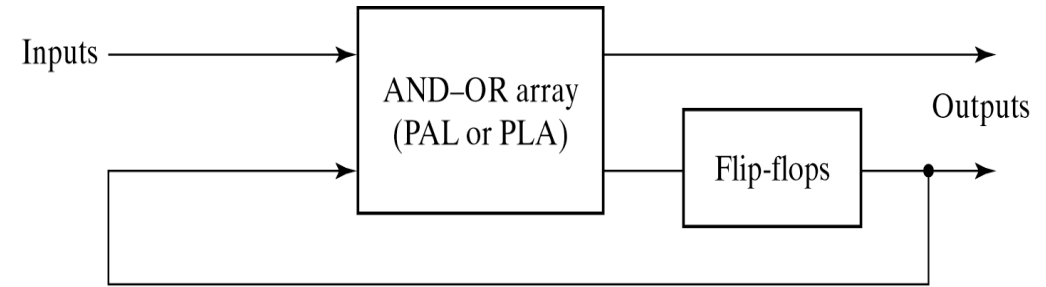
$F_1 = A'B' + A'C' + B'C'$   
 $F_1 = (AB + AC + BC)'$

|   |   |    |    |    |    |
|---|---|----|----|----|----|
|   |   | BC |    | B  |    |
|   |   | 00 | 01 | 11 | 10 |
| A | 0 | 1  | 0  | 0  | 0  |
|   | 1 | 0  | 1  | 1  | 1  |

C

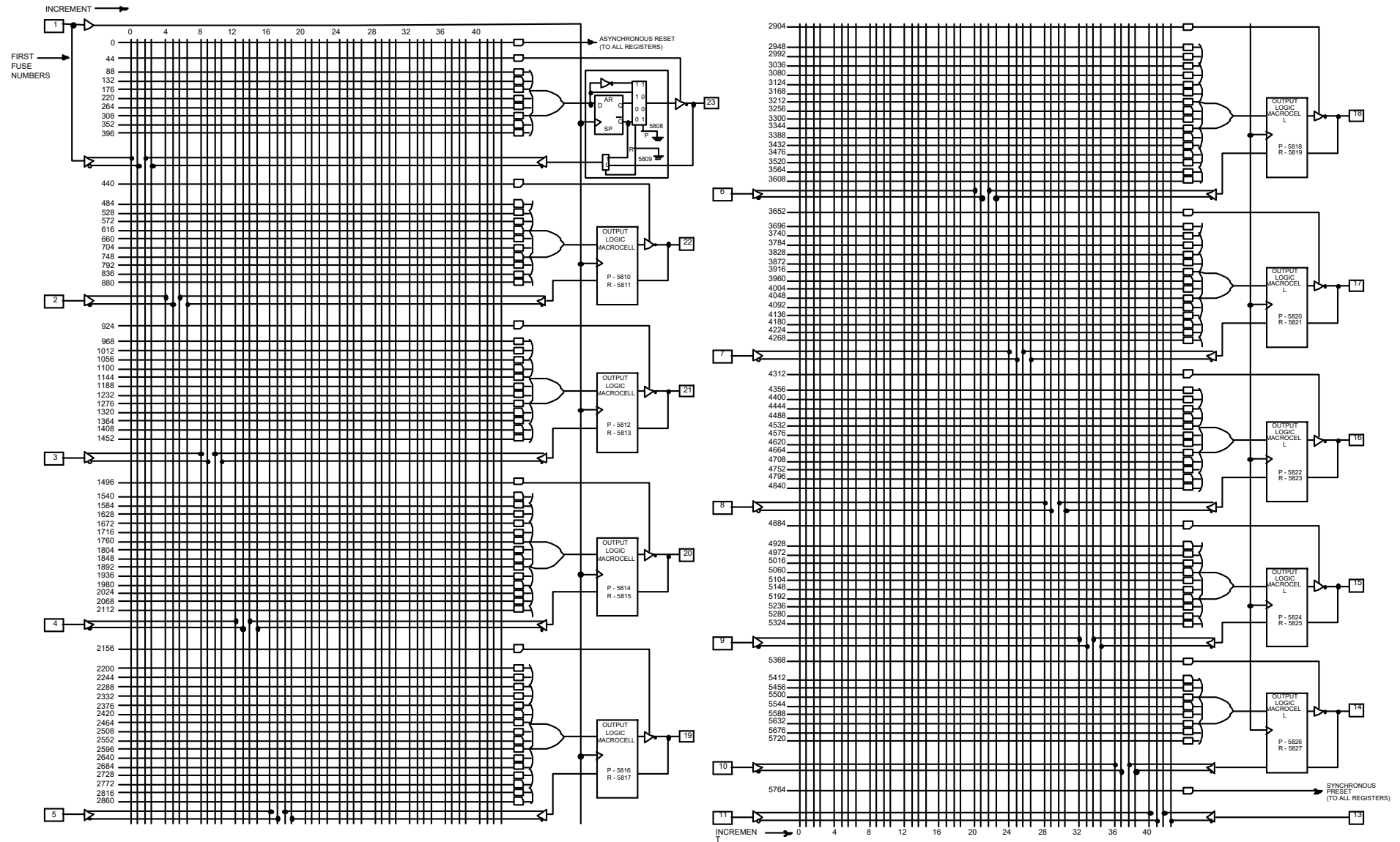
$F_2 = AB + AC + A'B'C'$   
 $F_2 = (A'C + A'B + AB'C')'$

- **Combinational PLD + Flip-flop**
  - **Combinational PLD can be PLA, but mostly PAL**
    - » dedicated product terms for next-state logic
  - **Flip-flop is mostly 1-input D flip-flop**
  - **Good for finite-state machine**
- **Also known as Registered PAL or Macrocell**
- **Design of finite-state machine with Sequential PLD**
  - **Design of next-state and output logic**
    - » follow the same design procedure of PAL
  - **Selection of flip-flops**
    - » flip-flops are given
- » **All the logic cells has the same clock and output enable**



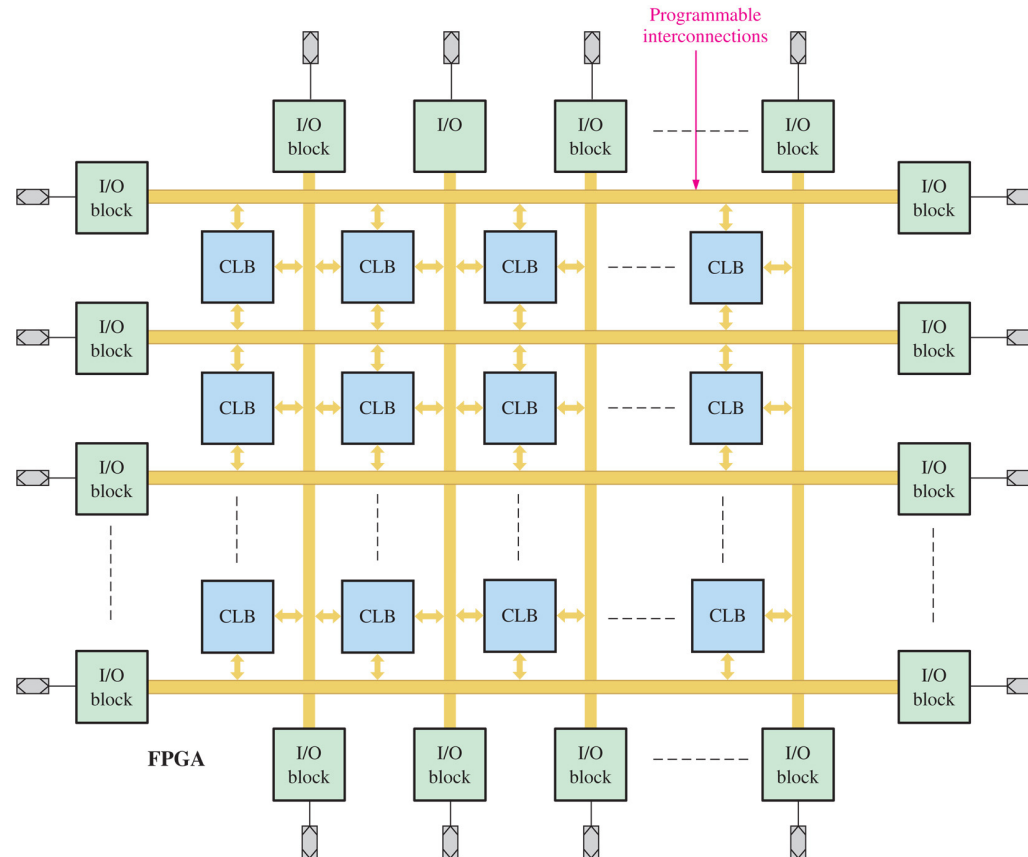


# Example of SPLD:P22V10 PAL, the Standard of Sequential PLD



## Basic structure of an FPGA.

CLB is configurable logic block, also known as logic array block (LAB).



## The three basic elements in an FPGA:

- the configurable logic block (CLB),
- the inter-connections, and
- the input/output (I/O) blocks,

The I/O blocks provide individually selectable input, output, or bidirectional access to the outside world.

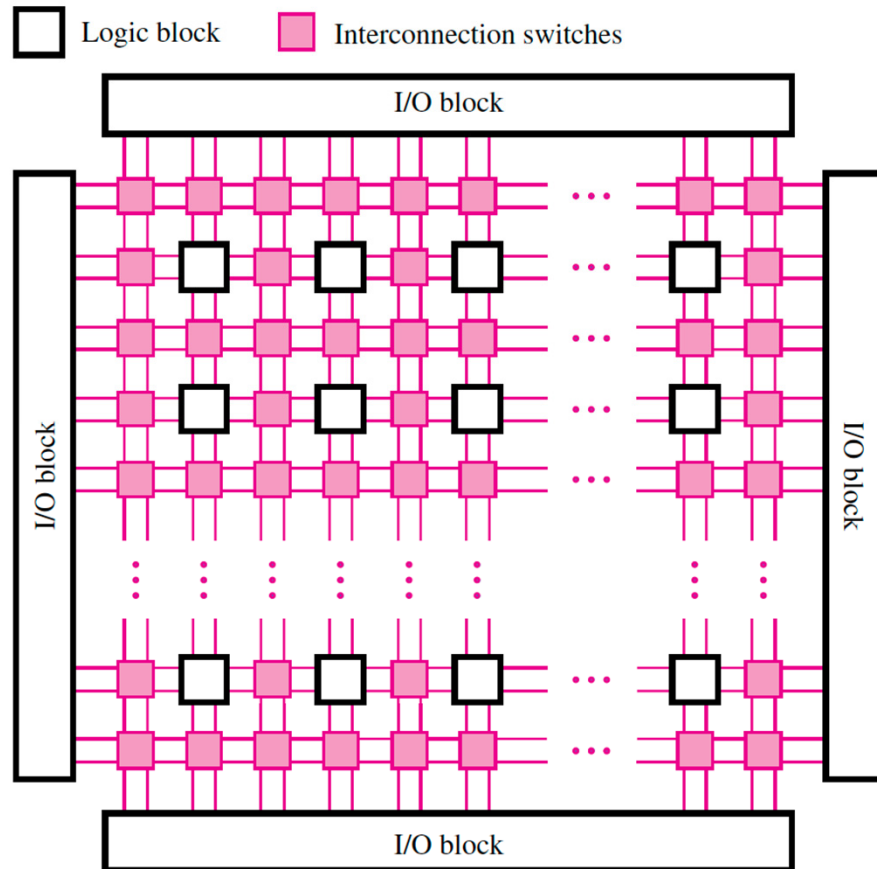
The distributed matrix of programmable interconnections provide for interconnection of the CLBs and connection to inputs and outputs.

Large FPGAs can have tens of thousands of CLBs in addition to memory and other resources.

# Field Programmable Gate Array

**Basic structure of an FPGA.**

**CLB is configurable logic block, also known as logic array block (LAB).**



**The three basic elements in an FPGA:**

- the configurable logic block (CLB),
- the inter-connections, and
- the input/output (I/O) blocks,

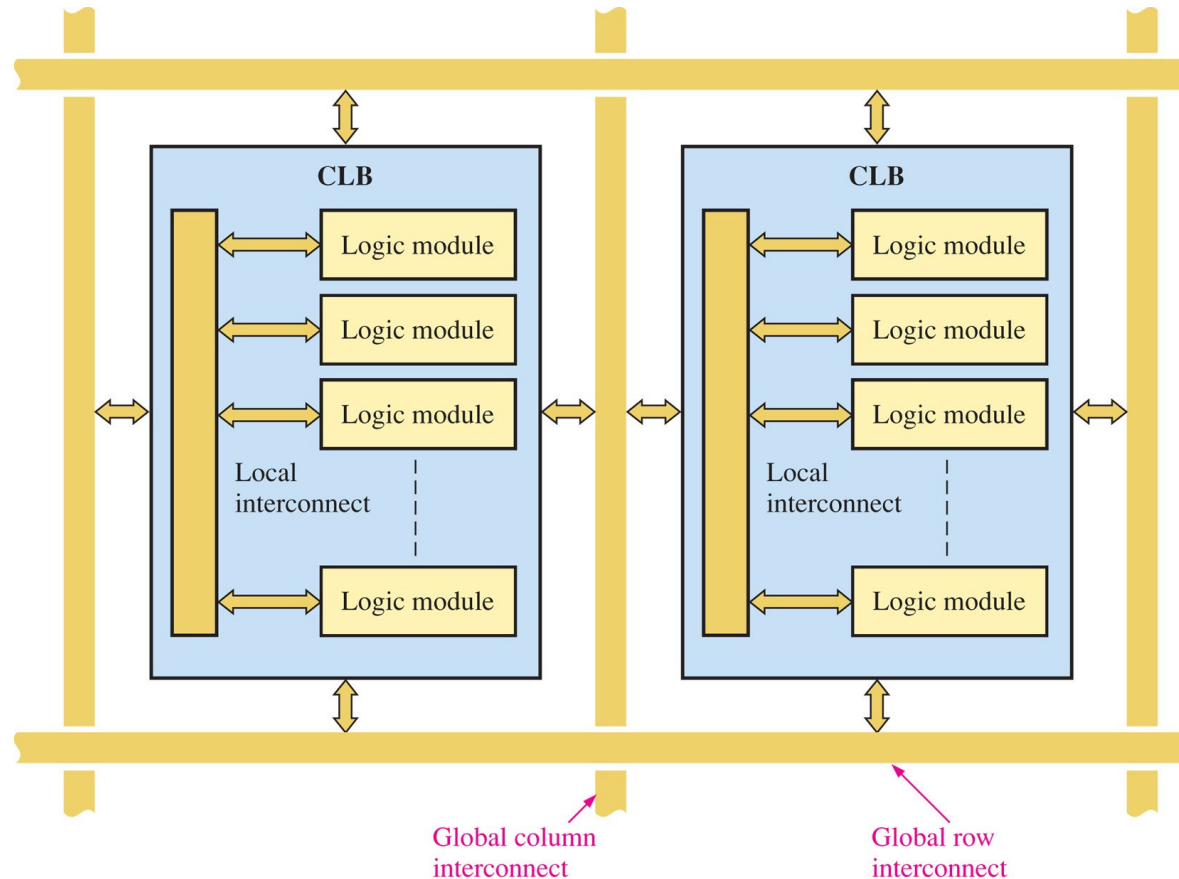
Red boxes adjacent to logic blocks hold switches that connect the logic block input and output terminals to the interconnection wires,

Red boxes that are diagonally between logic blocks connect one interconnection wire to another (such as a vertical wire to a horizontal wire).

Programmable connections also exist between the I/O blocks and the interconnection wires.

- **FPGAs range in density, power consumption, supply voltage, speed, and to some degree vary in architecture.**
- **FPGAs are reprogrammable and use SRAM or antifuse process technology for the programmable links.**
  - **SRAM or static RAM: Static random-access memory (static RAM or SRAM) is a type of random-access memory (RAM) that uses a flip-flop to store each bit. SRAM is volatile memory; data is lost when power is removed.**
- **Densities can range from hundreds of logic modules to hundreds of thousands of logic modules in packages with up to over 1,000 pins.**
- **DC supply voltages are typically 1.8 V to 5 V, depending on the specific device.**

# Basic configurable logic blocks (CLBs) within the global row/column programmable interconnects



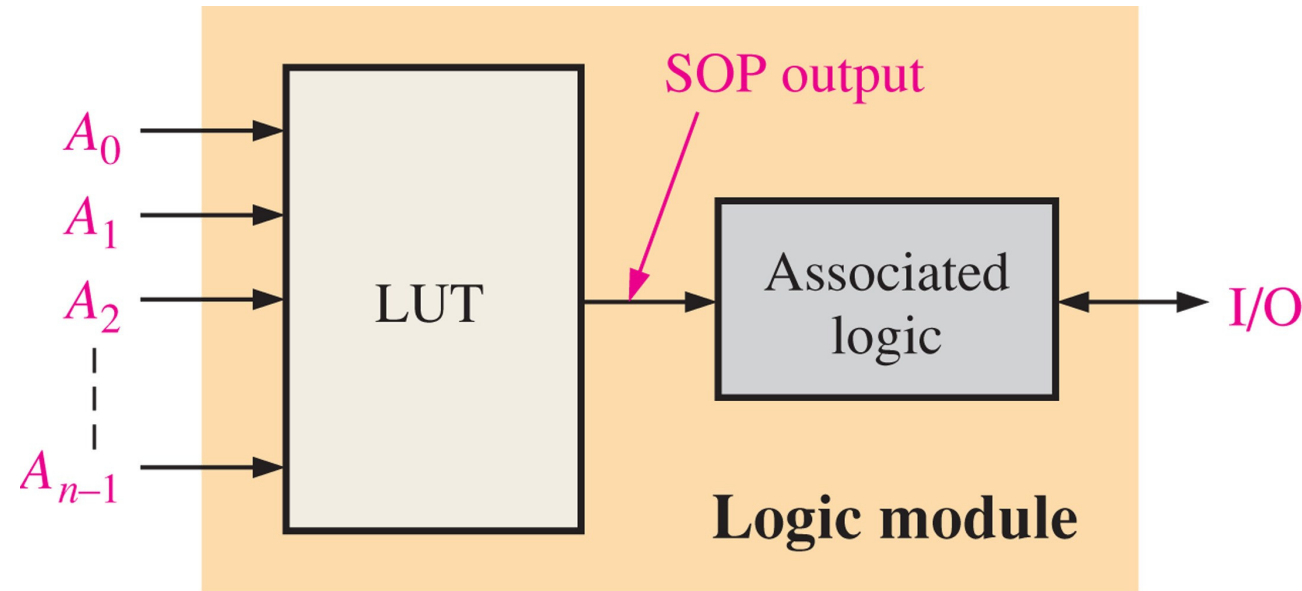
**Each CLB is made up of multiple smaller logic modules and a local programmable interconnect that is used to connect logic modules within the CLB**

# Basic block diagram of a logic module in an FPGA



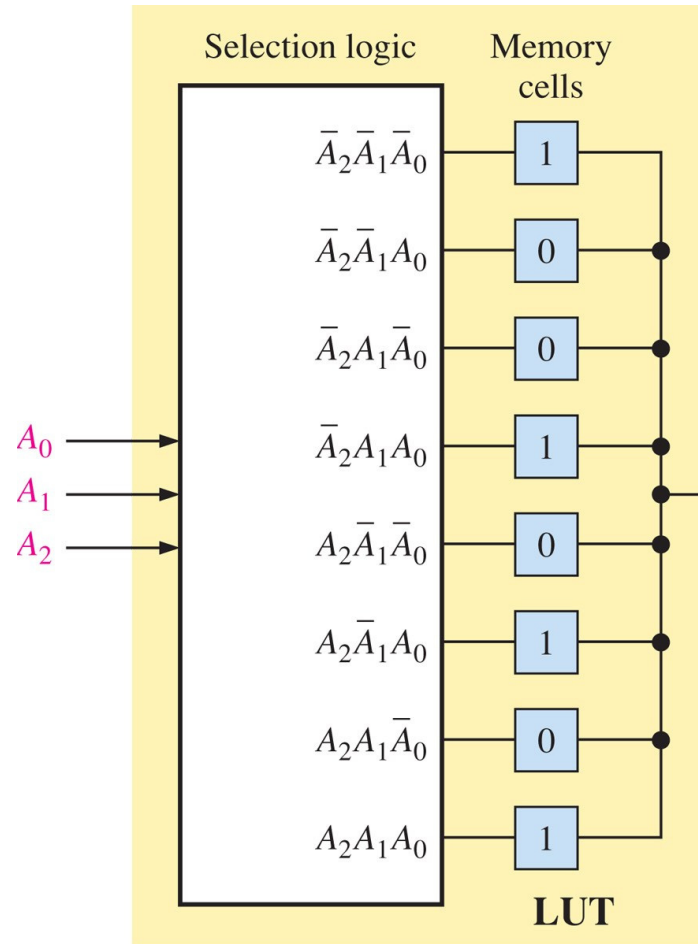
THE HONG KONG  
UNIVERSITY OF SCIENCE  
AND TECHNOLOGY

- A logic module in an FPGA logic block can be configured for combinational logic, registered logic, or a combination of both.
- A flip-flop is part of the associated logic and is used for registered logic.
- An LUT (look-up table) is a type of memory that is programmable and used to generate SOP combinational logic functions.



# The basic concept of an LUT programmed for a particular SOP output

Generally, the organization of an LUT consists of a number of memory cells equal to  $2^n$ , where  $n$  is the number of input variables



- three inputs can select up to eight memory cells, so an LUT with three input variables can produce an SOP expression with up to eight product terms.
- A pattern of 1s and 0s can be programmed into the LUT memory cells for a specified SOP function.
- Each 1 means the associated product term appears in the SOP output, and each 0 means that the associated product term does not appear in the SOP output.

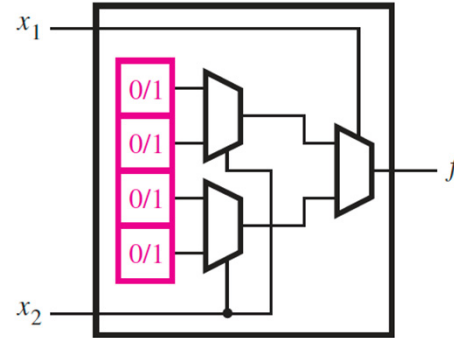
$$\bar{A}_2\bar{A}_1\bar{A}_0 + \bar{A}_2A_1A_0 + A_2\bar{A}_1A_0 + A_2A_1A_0$$

# Implementation of an LUT using Multiplexers.



THE HONG KONG  
UNIVERSITY OF SCIENCE  
AND TECHNOLOGY

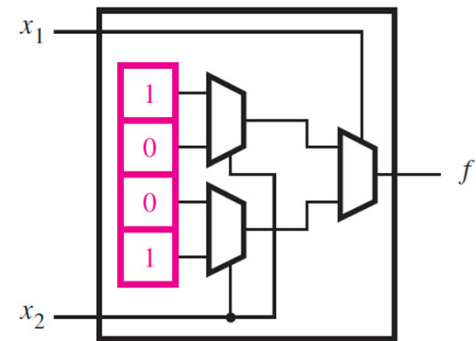
## A two-input LUT



(a) Circuit for a two-input LUT

| $x_1$ | $x_2$ | $f_1$ |
|-------|-------|-------|
| 0     | 0     | 1     |
| 0     | 1     | 0     |
| 1     | 0     | 0     |
| 1     | 1     | 1     |

(b)  $f_1 = \bar{x}_1\bar{x}_2 + x_1x_2$

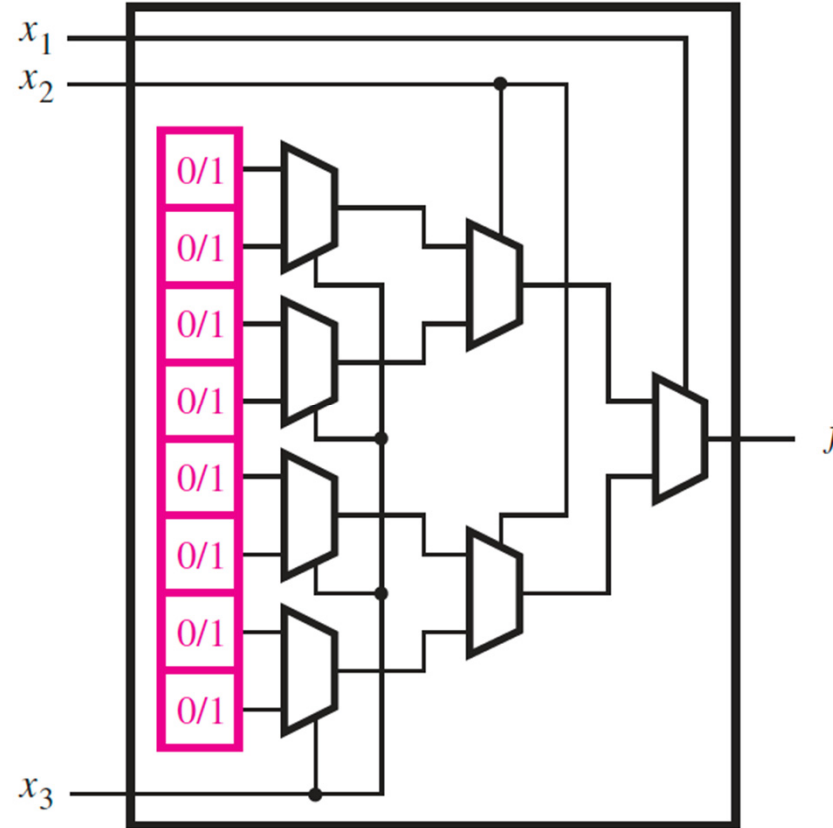


(c) Storage cell contents in the LUT



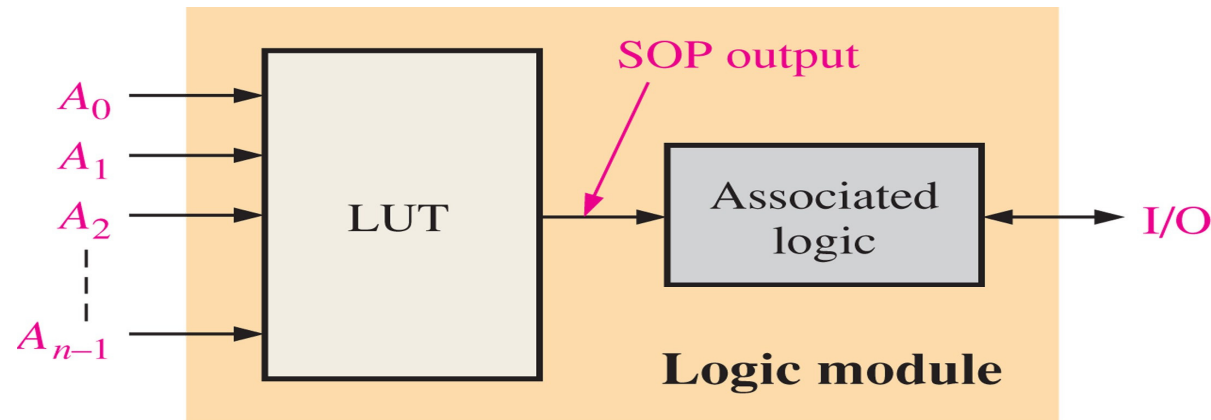
# Implementation of an LUT using Multiplexers.

## A three-input LUT

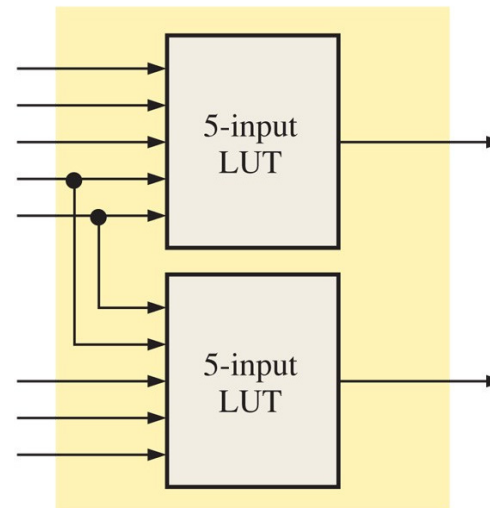
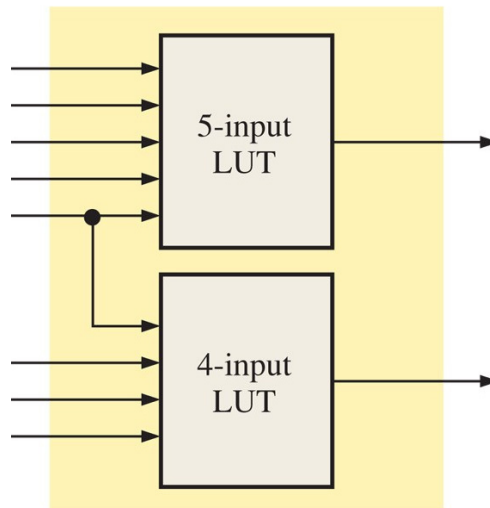
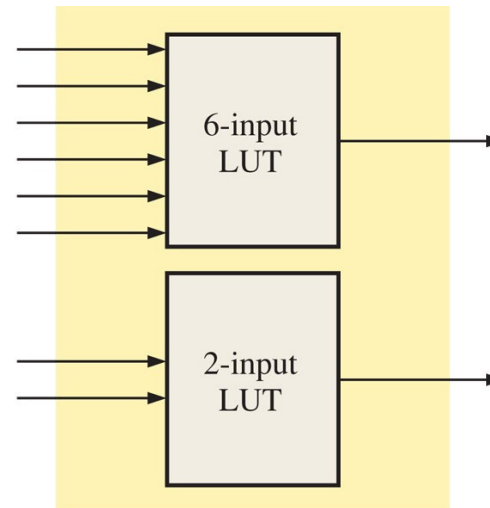
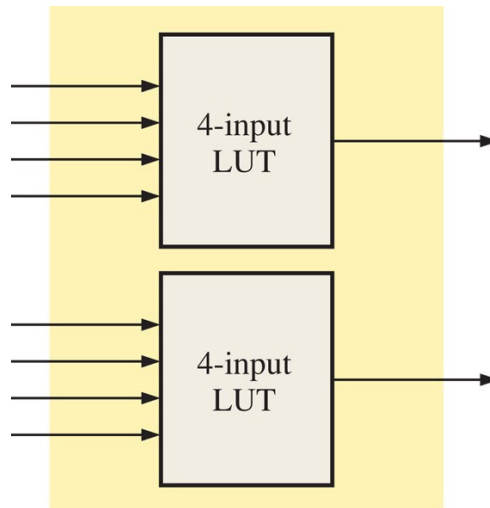


# Operating modes of a Logic Module

- Typically, a logic module (LM) can be programmed for the following modes of operation:
  - Normal mode (primarily used for generating combinational logic functions)
  - Extended LUT mode
  - Arithmetic mode
  - Shared arithmetic mode
- In addition to these four modes, a logic module can be utilized as a register chain to create counters and shift registers.



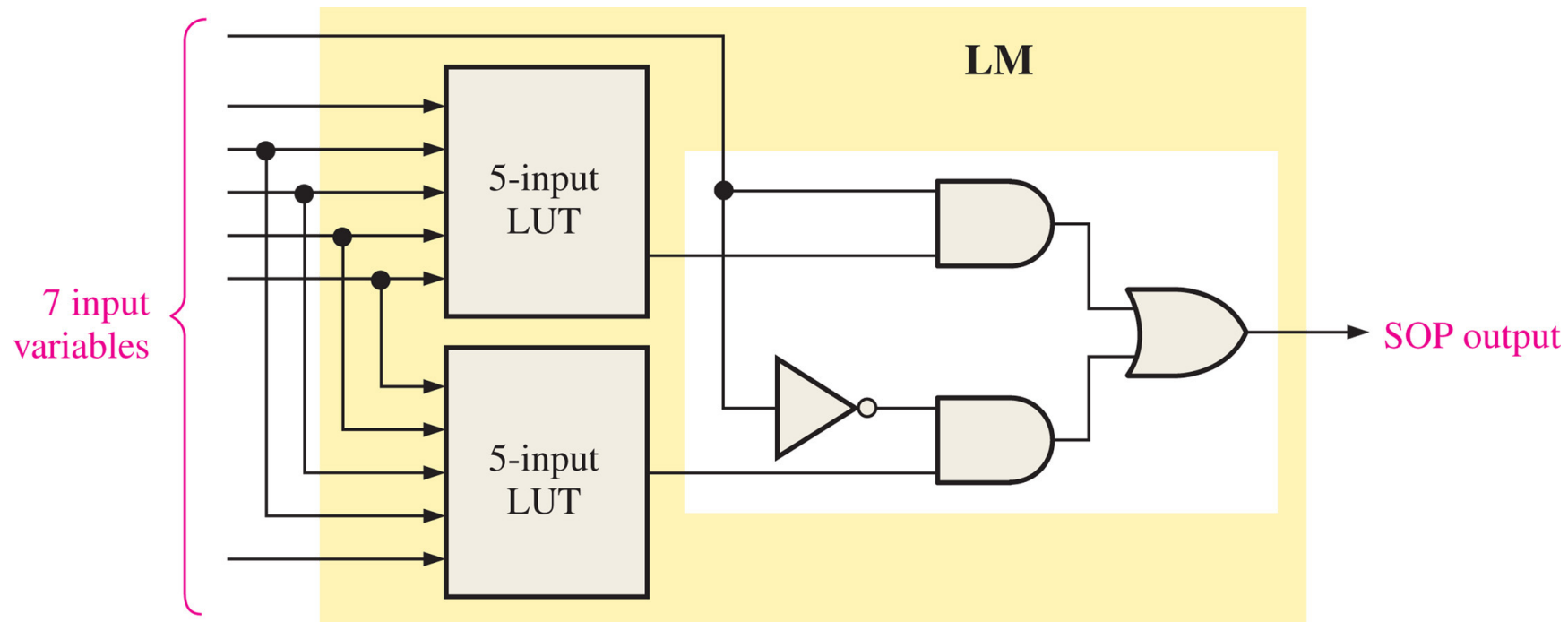
# Examples of 4 possible LUT configurations in a logic module (LM) in the normal mode



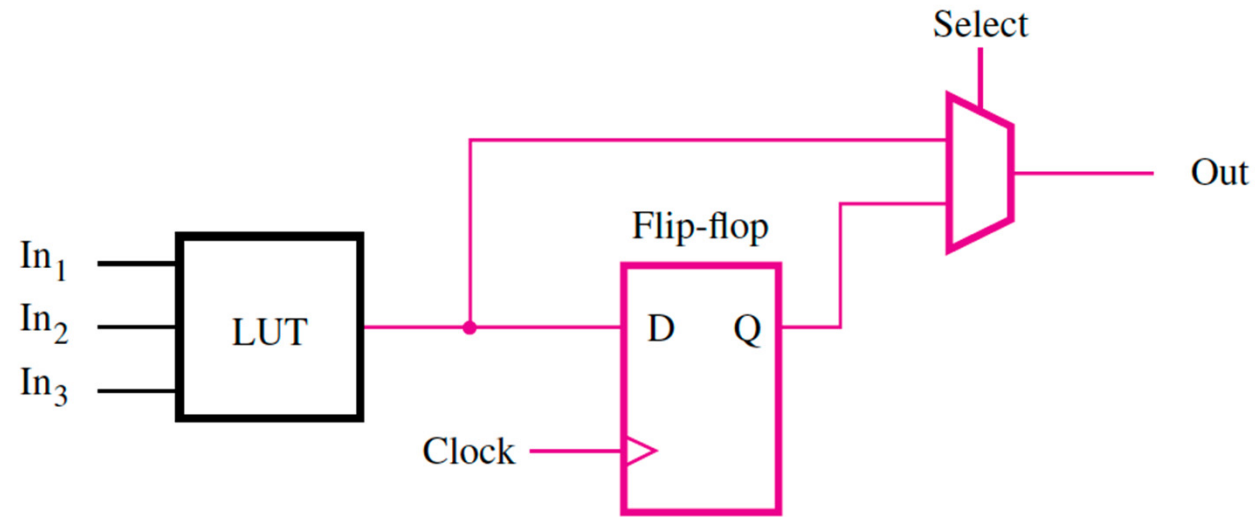
- A logic module can implement one or two combinational output functions with its two LUTs.
- Generally, two SOP functions, each with four variables or less, can be implemented in an LM without sharing inputs.
- By sharing inputs, you can have any combination of a total of eight inputs up to a maximum of six inputs for each LUT. In the normal mode, you are limited to 6-variable SOP functions.

# Expansion of a logic module (LM) to produce a 7-variable SOP function in the extended LUT mode

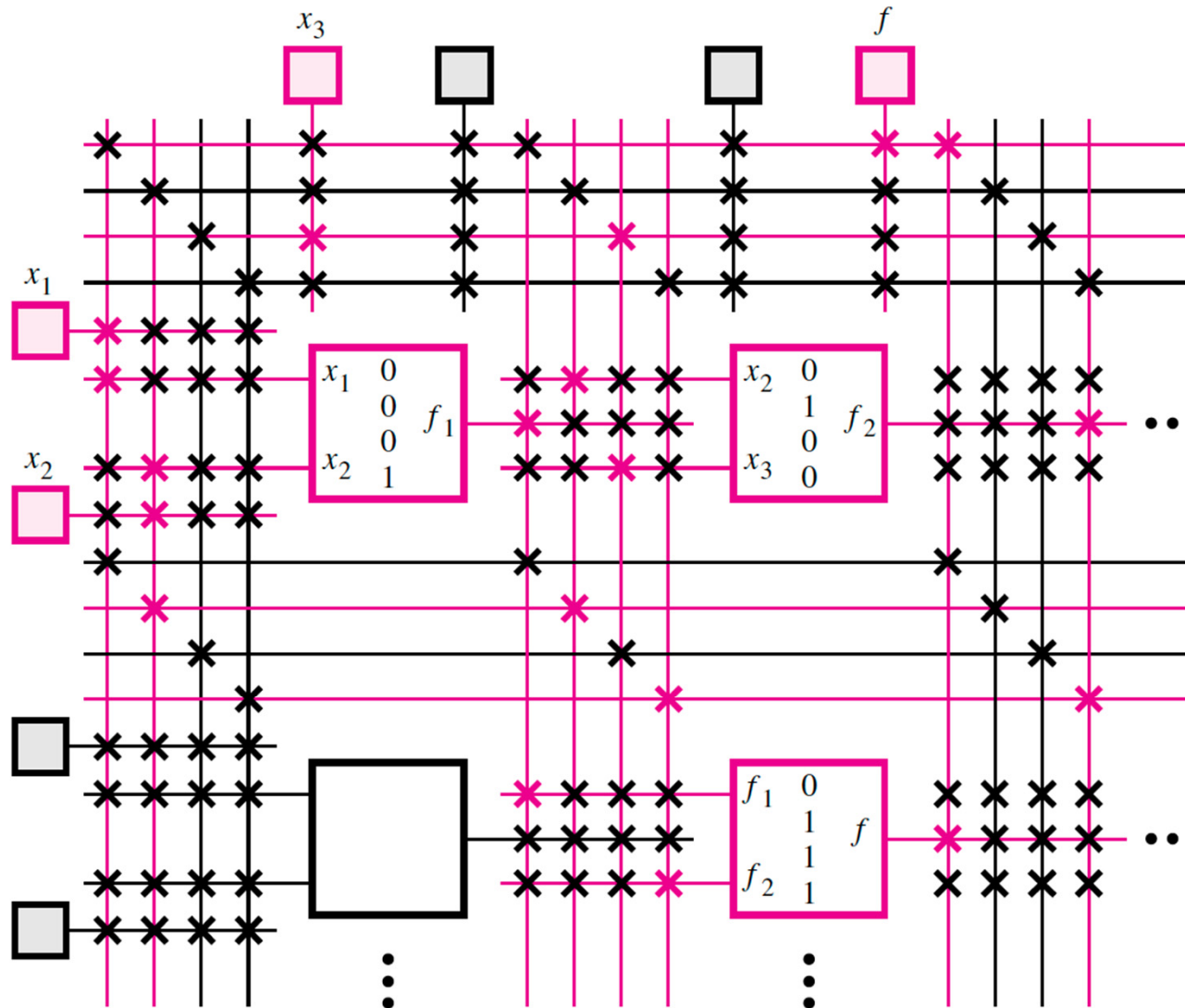
The extended LUT mode allows expansion to a 7-variable function, as illustrated here. The multiplexer formed by the AND-OR circuit with a complemented input is part of the dedicated logic in a logic module



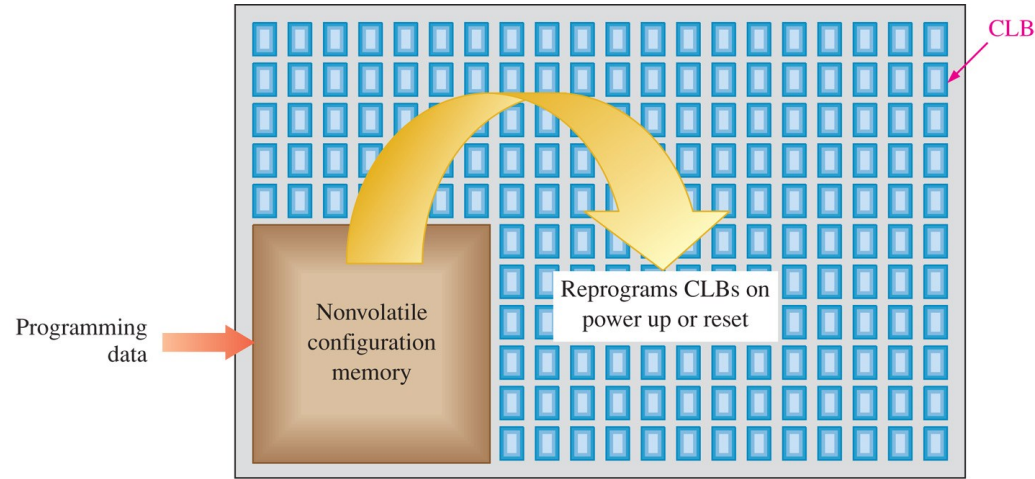
# Inclusion of a flip-flop in an FPGA logic block



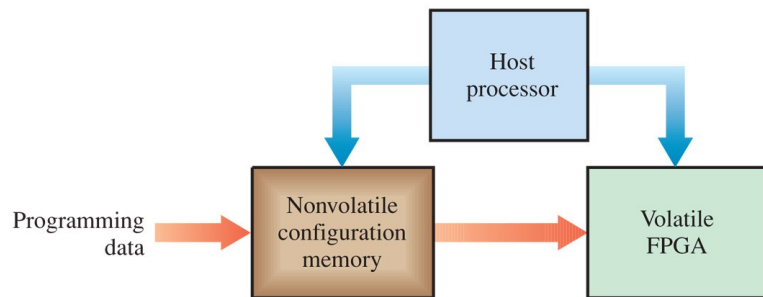
# A section of a programmed FPGA



# Basic concepts of volatile FPGA configurations



(a) Volatile FPGA with on-the-chip nonvolatile configuration memory



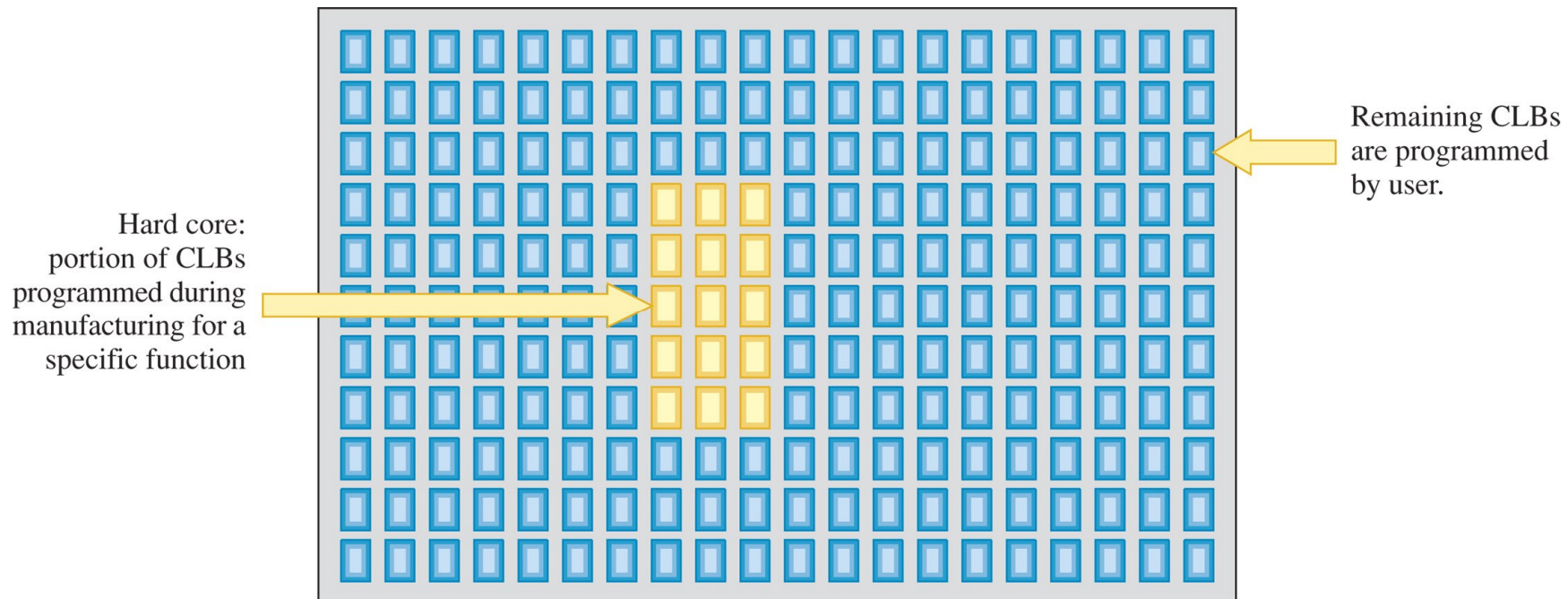
(b) Volatile FPGA with on-board memory and host processor

**FPGAs are either nonvolatile because they are based on antifuse technology or they are volatile because they are based on SRAM technology**

**SRAM-based FPGAs include either a nonvolatile configuration memory embedded on the chip to store the program data and reconfigure the device each time power is turned back on or they use an external memory with data transfer controlled by a host processor**

# A hard-core function embedded in an FPGA

- FPGAs are available that also contain hard-core logic put in by the manufacturer to provide a specific function and that cannot be reprogrammed (a small microprocessor)
- If the embedded function has programmable features, it is known as a soft-core function.
- An advantage of the hard-core approach is that the same design can be implemented using much less of the available capacity of the FPGA than if the user programmed it in the field, resulting in less space on the chip (“real estate”) and less development time for the user.
- The disadvantage of the hard core is that the specifications are fixed during manufacturing and the customer must be able to use the hard-core logic “as is.” It cannot be changed later

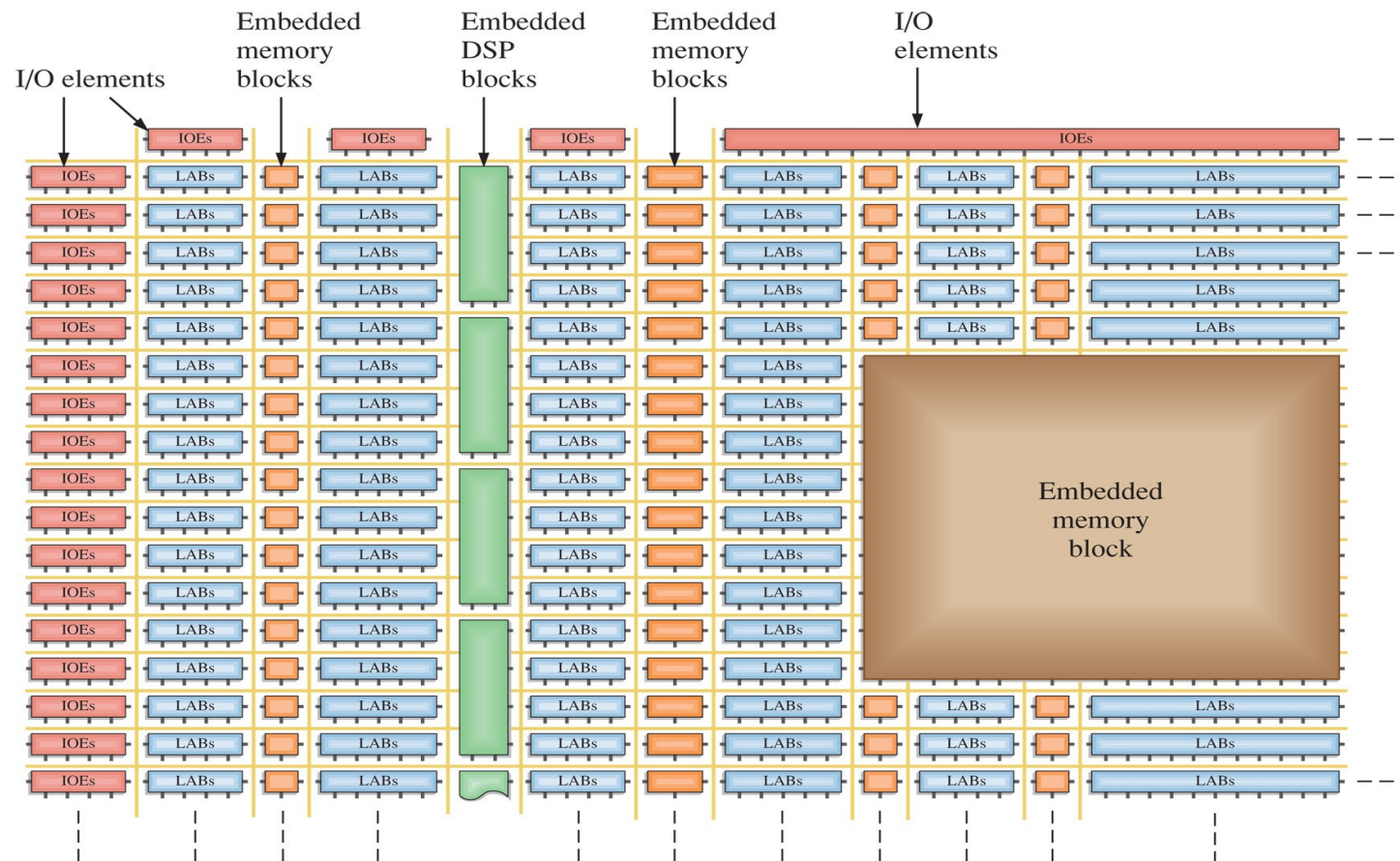




- **Hard cores are generally available for functions that are commonly used in digital systems, such as a microprocessor, standard input/output interfaces, and digital signal processors.**
- **More than one hard-core function can be programmed in an FPGA.**
- **Hard core designs are generally developed by and are the property of the FPGA manufacturer. Designs owned by the manufacturer are termed intellectual property (IP).**
- **A company usually lists the types of intellectual property that are available on its website. Some intellectual properties are a mix of hard core and soft core. A processor that has some flexibility in the selection and adjustment of certain parameters by the user is an example.**
- **Those FPGAs containing either or both hard-core and soft-core embedded processors and other functions are known as platform FPGAs because they can be used to implement an entire system without the need for external support devices.**

# Example FPGA block diagram

The FPGA contains embedded memory functions as well as digital signal processing (DSP) functions. DSP functions, such as digital filters, are commonly used in many systems. The embedded blocks are arranged throughout the FPGA interconnection matrix and input/output elements (IOEs) are placed around the FPGA perimeter.



**TABLE 10–3**

FPGA manufacturers.

| Manufacturer | Series Name(s)                                                | Design Software             | Website         |
|--------------|---------------------------------------------------------------|-----------------------------|-----------------|
| Altera       | Stratix<br>Aria<br>Cyclone                                    | Quartus II                  | Altera.com      |
| Xilinx       | Spartan<br>Artix<br>Kintex<br>Virtex                          | ISE Design Suite            | Xilinx.com      |
| Lattice      | iCE40<br>MachX02<br>Lattice ECP3<br>LatticeXP2<br>LatticeGC/M | Lattice Diamond<br>iCEcube2 | Latticesemi.com |
| Atmel        | AT40                                                          | IDS                         | Atmel.com       |

**TABLE 10-4**

Selected FPGA parameters.

| Feature              | Range                    |
|----------------------|--------------------------|
| Number of LEs        | 1,500–813,000            |
| Number of CLBs       | 26–359,000               |
| Embedded memory      | 26 kb–63 Mb              |
| Number of I/Os       | 18–1200                  |
| DC operating voltage | 1.8 V, 2.5 V, 3.3 V, 5 V |



香港科技大學  
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

*Thank you for listening !*

